COSTPILOT · TECHNICAL SPECIFICATION

Build Blueprint

Architecture, contracts, data model, engine rulebook, and delivery gates for the management-accounting copilot.

Version	1.0 — November 2026
Status	Build-ready for Phase 1; extension contracts defined for Phase 2
Audience	Engineering lead, backend and frontend engineers, cost-accounting SME, product owner
Companions	CostPilot White Paper; Phase 1 Plan; Phase 2 Plan
Scope	Phase 1 in build detail; Phase 2 interfaces, schema extensions, and sequencing

This document fixes the structural decisions that are expensive to reverse: the trust boundary between the language model and the calculation engine, the cost data model, the audit trace, and the rule registry. Visual design, pricing, and go-to-market remain outside its scope.

CONTENTS

1. Purpose and scope
What this document decides, and what it deliberately leaves open

2. Design invariants
Ten rules that every component and every test must enforce

3. System architecture
Runtime shape, request lifecycle, deployment topology

4. The LLM contract
Planner schema, tool registry, confirmation, slot-based narration

5. Cost data layer
Dimensional model, account registry, quality gates, snapshots

6. Deterministic engine
Rulebook, calculation results, worked examples, edge cases

7. Audit and explainability
Trace object, hash chain, re-performance, analysis-card requirements

8. API and interface contracts
Endpoints, streaming events, error taxonomy, exports

9. Repository and engineering standards
Monorepo layout, stack decisions, CI gates, security

10. Test and evaluation strategy
Golden cases, property tests, LLM evals, performance budgets

11. Delivery roadmap
M0–M11, dependencies, exit gates, team shape

12. Risks and open decisions
Build risks, defaults needing sign-off, out-of-scope contract

13. Appendices
Glossary, schemas, rulebook template, source traceability

1 Purpose and scope

CostPilot already has three documents that establish intent. The White Paper argues why management accounting is the unattended territory and what the product is. The Phase 1 Plan defines modules, milestones, and success criteria. The Phase 2 Plan adds activity-based costing, budgeting, performance measurement, connectors, and monitoring. This blueprint is the fourth document: it translates those documents into the contracts an engineering team can build against.

It is deliberately opinionated. Where the phase plans leave a technical choice open, this document names a default, records the trade-off, and assigns the decision an owner. That is not an attempt to freeze the product. It is an attempt to freeze the parts that become expensive to change after the first pilot: the data model, the boundary between model and engine, the audit trace, and the rulebook.

1.1 What “blueprint” means here

Not a UI specification and not a sprint plan. The blueprint covers the structural layer:

- **Contracts** between the user interface, the planner model, the tool registry, the calculation engine, and the audit service.
- **Schemas** for the cost data layer, the rulebook, the calculation trace, and the API payloads.
- **Policy** for sign conventions, rounding, units, missing data, capacity assumptions, and refusal behaviour.
- **Gates** that decide when a milestone is actually complete and when an answer is allowed to reach a user.

Interface copy, chart styling, onboarding, and the precise wording of prompts will change weekly. The decisions here should not.

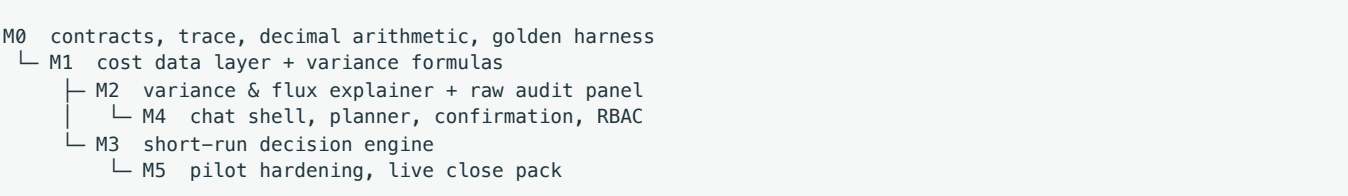
1.2 Phase 1 is done when

> A cost analyst can load one month of GL actuals, standard costs, budgets, and volumes for a three-product, two-plant manufacturer; ask a variance question or one of seven short-run decision questions; and receive an answer whose every figure is computed by the engine, traceable to source records and rule IDs, and re-performable by a controller. The same question against the same snapshot returns the same numbers, byte for byte. The working paper exports to Excel without a parallel spreadsheet.

That statement is narrower than the product vision and broader than a demo. It requires the data layer, the variance engine, the decision engine, the planner, the audit surface, and the export path to work together on one realistic dataset. It does not require ABC, ERP connectors, or multi-entity consolidation.

1.3 Build order

The dependency chain is strict. Data and formulas come first because every later module consumes their contracts. The variance explainer comes next because it exercises the full trace and narration path on a familiar finance output. The decision engine follows, because its outputs are the sales demo and because it stress-tests assumption handling. The conversational shell comes last, not because it is unimportant but because a planner without trustworthy tools is a demo, not a product.



Phase 2 follows the same pattern: connector foundation first, because ABC driver data, alerting, and scorecard KPIs all depend on richer ingestion; ABC second, because it is the strongest value demonstration; then monitoring, budgeting, performance, and group reporting. Section 11 gives the full sequence with exit gates.

1.4 Deliberate non-goals

Phase 1 is not a general ledger, an ERP, a report writer, a forecasting system, or a general-purpose finance chatbot. It does not write back to any system. It does not support multi-currency translation, intercompany elimination, or ABC. It does not infer account classifications silently and it does not answer a question whose data is missing by estimating a plausible number. Section 12.3 restates these as a scope contract that change control must explicitly reopen.

2 Design invariants

An invariant is a property that must hold for every run, in every environment, for the life of the product. Each one below has an enforcement mechanism and a test. If an invariant cannot be enforced mechanically, it is a wish, not an invariant. The ten that follow are the ones most likely to be violated under delivery pressure.

#	Invariant	Enforcement	Test
I1	**Engine authority.** No number in an answer originates from the language model.	Narration is slot-based: the model receives fact IDs and may emit only `{{id|format}}` placeholders. Raw digits and spelled-out quantities are rejected before rendering. Verdict sentences are deterministic templates selected by the engine, not written by the model.	Provenance validator; adversarial prompt suite; CI check that planner prompts contain no arithmetic results.
I2	**Basis is always labelled.** Actual, budget, flexible-budget, standard, and forecast figures are never mixed without a visible basis and period.	Every fact row carries `basis`, `period_id`, `snapshot_id`, and `unit`. The renderer prints basis chips in the card header and on every table.	Contract tests reject facts without basis; UI snapshot tests assert the chip.
I3	**Snapshot determinism.** Same question against the same snapshot, engine, and rulebook returns the same numbers.	Run pins snapshot hash, engine semver, rulebook version, registry version, prompt version, and model ID. Data snapshots are immutable. Execution order is stable; no unordered iteration.	Re-performance test on every completed run sample; nightly determinism replay.
I4	**Provenance completeness.** Every figure links to a calculation step and every step links to source records.	The `CalcResult` object cannot be constructed without step IDs and source refs. The validator rejects orphan facts. The audit UI renders the chain.	Trace completeness test: zero facts without a step; zero steps without a rule ID.
I5	**Assumptions are objects.** Capacity, idle time, avoidable cost, and policy choices are stated, confirmed, and versioned. No silent default.	Planner arguments carry `source` and `confirm` flags. The orchestrator blocks execution until every inferred or policy-bearing argument is confirmed. Edits create a new plan version.	Assumption-gate tests; runs with unconfirmed assumptions must fail closed.
I6	**Read-only.** CostPilot never writes to a ledger, ERP, or source system.	Connectors use least-privilege read credentials; no write endpoints exist; exports are copies. Network policy denies write APIs.	Architecture test scans connector code for write verbs; credentials audit.
I7	**Rule fidelity.** Every formula maps one-to-one to a registered rule with a source citation and test cases.	Rule registry is the only path to arithmetic. UI, planner, and API cannot contain formulas. Rule changes require version bumps and golden-case updates.	Golden corpus reproduces every source worked example exactly; import-boundary lint.
I8	**Scope isolation.** Entity, plant, and role restrictions are applied inside the query, not after it.	Query compiler injects row-scope predicates before SQL generation. No endpoint returns unfiltered rows. DuckDB files are per tenant.	Authorization matrix tests; attempted cross-scope query returns empty plus an access error, never data.

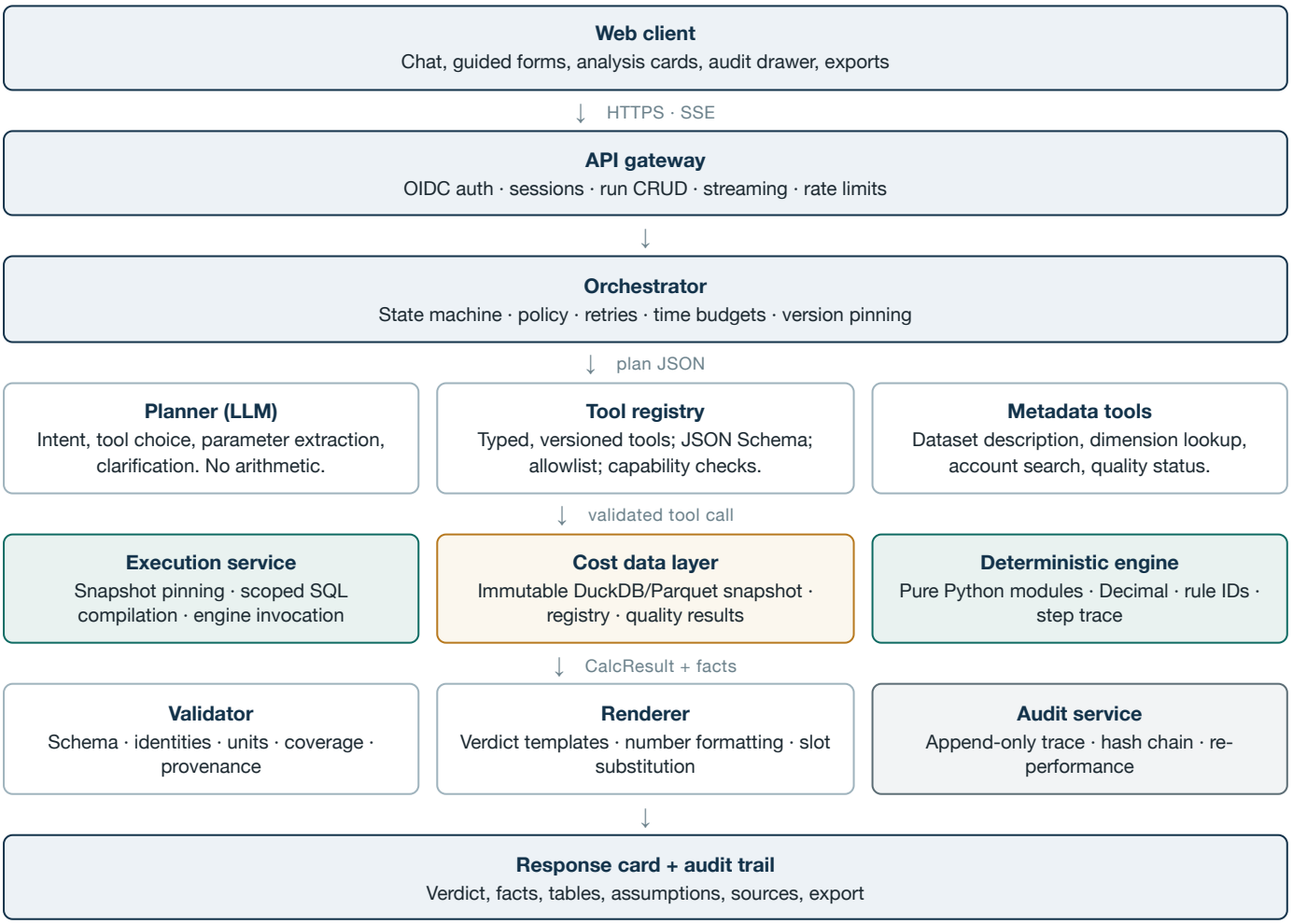
#	Invariant	Enforcement	Test
I9	**Honest refusal with a path.** Missing data produces a specific remediation, never a guess and never a dead end.	Every error carries `cause`, `what_was_checked`, and a `remediation[]` array of executable next steps: fix a tag, upload a file, confirm an assumption, choose a period, view the trace.	Error-catalogue tests assert remediation is present and actionable for every code.
I10	**Explainability is product surface.** The audit view is a first-class panel, not a debug toggle.	The analysis card cannot render without an audit affordance. Export includes the trace. Acceptance tests require a controller to re-perform from the screen.	UX acceptance checklist; pilot controller sign-off.

The invariant most likely to erode is I1. It is easy to let a model "just tidy" a number, restate a percentage, or convert units in prose. Every such convenience reintroduces the failure the product exists to prevent. The slot protocol in Section 4.6 is intentionally rigid; do not relax it for a nicer sentence.

3 System architecture

The runtime is a deterministic orchestrator around a language model, not an autonomous agent around a database. The distinction matters. The model proposes a plan; the orchestrator decides whether the plan is admissible, what it is allowed to touch, and when the result is fit to show. The engine computes; the renderer formats; the audit service records. No component is trusted to do another's job.

3.1 Runtime shape



3.2 Component responsibilities

Component	Owns	Must never
Web client	Conversation, guided forms, analysis-card rendering, assumption chips, audit drawer, export controls.	Compute, round, or derive a figure; hide the basis; render a number absent from the response payload.
API gateway	Authentication, session ownership, run access, streaming, rate limits, request validation.	Apply business logic or bypass orchestrator policy.
Orchestrator	State machine, plan admission, confirmation gate, version pinning, retries, timeouts, fallbacks.	Let the model call the engine directly; proceed with an unconfirmed policy assumption.
Planner	Intent classification, tool selection, parameter extraction, one focused clarification question, assumption proposals.	Compute, estimate, convert units, invent IDs, narrate results, or see raw ledgers.
Tool registry	Tool schemas, versions, capability requirements, compatibility, deprecation.	Expose an untyped or unversioned tool; permit arbitrary SQL.
Execution service	Resolving arguments to dimensions, compiling scoped queries, invoking engine modules, assembling the result envelope.	Guess a dimension match; read outside the pinned snapshot.
Cost data layer	Canonical schema, account registry, ingestion, quality gates, immutable snapshots, period state.	Mutate a frozen snapshot; silently coerce an untagged account.
Deterministic engine	All arithmetic: variances, flexible budgets, CVP, differential analysis, costing bridge, ABC, budgets, performance.	Perform I/O, read the database, call the model, use floats, or hide a policy choice.
Validator	Schema, identity, unit, basis, coverage, assumption, and provenance checks. Decides whether an answer may ship.	Warn and continue on a blocking failure.
Renderer	Deterministic templates for verdicts, tables, formatting, slot substitution, export layout.	Let the model supply a raw numeral or override a verdict.
Audit service	Immutable run record, hash chain, trace retrieval, re-performance, retention.	Update or delete a sealed run; store secrets.

3.3 Request lifecycle

1. **Receive.** The API authenticates the user, resolves tenant and row scope, and appends the message to the session.
2. **Ground.** The orchestrator builds a planner context: a capability card (datasets, periods, dimensions, supported analyses, data-quality status), a conversation summary, and references to prior run IDs. No raw ledger rows.
3. **Plan.** The planner returns a plan JSON: intent, tool, arguments, assumptions, missing fields, and at most one clarification question.
4. **Admit.** The orchestrator validates the plan against the tool schema and policy. Unknown tools, out-of-scope dimensions, arithmetic fields, or unregistered decision types are rejected.
5. **Confirm.** If any argument is inferred, defaulted, or policy-bearing, the user sees a parameter sheet with value, source, and edit control. Execution waits. Edits create a new plan version.
6. **Pin.** The orchestrator creates a run and pins the snapshot, registry, engine, rulebook, prompt, and model versions. The input hash is computed.
7. **Execute.** The execution service resolves dimensions, compiles scoped SQL, and calls the engine. Heavy scans run against pre-aggregated marts where possible.
8. **Compute.** The engine returns a `CalcResult` with steps, facts, assumptions, warnings, and source references. No prose yet.
9. **Validate.** Schema, identity, unit, coverage, basis, and provenance checks run. A blocking failure stops the run with a remediation card.
10. **Render.** The renderer selects the verdict template from the engine's decision output, requests a rationale from the model using fact slots only, validates the rationale, and substitutes formatted values.
11. **Seal.** The audit record is hashed and appended. The response streams to the client with the run ID.

12. **Follow up.** The card offers export, re-performance, a trace view, and suggested next questions generated from the capability card, not from the model's imagination.

> **No answer exists before step 9 passes.** Partial results, "rough" numbers, and provisional narratives are not sent. A failed validation produces an error card with the checked facts and a path forward, never a softened answer.

3.4 Two interaction lanes, one contract

Question lane

Free-text or voice-transcribed question. The planner selects a tool and extracts parameters. Best for experienced analysts who know what they want and for follow-ups such as "now assume the idle capacity is only 1,000 units."

Workflow lane

A guided form generated from the same tool JSON Schema. Required fields, enums, units, and confirmation are identical. Best for non-accountants, for auditability, and as the fallback when the model is unavailable.

The lanes must not diverge. Tool schemas are the single source of truth for fields, validation, and confirmation. The form renderer generates controls from schema annotations (`title`, `description`, `examples`, `x-unit`, `x-confirm`). A new workflow is added once, in the registry, and appears in both lanes.

3.5 Deployment topology

The pilot is single-tenant in a customer-controlled VPC or a managed private cloud. Shared multi-tenant DuckDB is prohibited: a snapshot directory contains one tenant's data and one role scope, and workers are pinned to a tenant for the duration of a job.

Service	Role	Pilot configuration
web	Next.js application; server-side session cookie; SSE proxy.	2 instances behind TLS termination.
api	FastAPI orchestrator and read APIs; no heavy computation in-process.	2–4 workers; request timeout 30 s; stream heartbeats.
worker	Ingestion, snapshot builds, exports, scheduled monitoring, re-performance jobs.	Redis-backed queue; one heavy job per worker; memory cap per DuckDB process.
Postgres	Metadata, registry, sessions, runs, audit records, mapping contracts.	Managed instance; PITR; 7-year audit retention.
Object storage	Raw uploads, canonical Parquet, snapshot manifests, exports.	Versioning and object lock for sealed snapshots and audit exports.
DuckDB	Query engine over immutable snapshot files; one database per snapshot build.	Local SSD or attached volume; read-only open; memory limit per worker.
LLM gateway	Egress allowlist, provider abstraction, zero-retention policy, token accounting.	Region-pinned endpoint; no training on customer data; request logging redacted.
Observability	OpenTelemetry traces, structured logs, metrics, alerting.	Run IDs propagated end to end; amounts excluded from logs by default.

Backups cover Postgres (point-in-time) and object storage (versioned). DuckDB files are rebuildable from Parquet and the manifest; they are cache, not the system of record. A snapshot is not "backed up" as a database file; it is verified by re-hashing its manifest and re-running its quality gates.

3.6 Centralized versus pluggable

Three things stay centralized because fragmentation would break trust: the **rule registry** (one formula set, one sign convention), the **trace format** (one audit schema), and the **validator** (one definition of shippable). Three things are pluggable because Phase 2 and customer environments demand it: **data connectors**, **cost models** (traditional versus

ABC/TDABC), and **renderers** (chat card, Excel, PDF). A connector may not introduce a new fact shape without a schema migration; a cost model may not bypass the rule registry.

4 The LLM contract

The planner is the only non-deterministic component in the answer path. It is therefore treated as an untrusted proposals engine: useful for language, inadmissible as an authority. The contract below is enforced by schemas and the orchestrator, not by prompt wording alone. A model that ignores the prompt should fail validation, not produce a plausible wrong answer.

4.1 What the model may and may not do

May	May not
Classify the question into a registered intent or return “unsupported”.	Answer a cost question from general knowledge or from numbers in the conversation.
Select one tool from the allowlist and populate its arguments.	Call arbitrary SQL, HTTP, filesystem, or unregistered tools.
Resolve names to IDs using metadata tools, returning candidates when ambiguous.	Invent an account, product, plant, period, or customer ID.
Extract operators and operands, such as “price down 10%” or “volume up 2,000 units”.	Apply the operator. The engine applies the change; the model passes the operand.
Propose assumptions with a source, confidence, and confirmation flag.	Choose an accounting policy, capacity assumption, or avoidable-cost treatment silently.
Ask one consolidated clarification question when required fields are missing or ambiguous.	Ask a sequence of questions it could have combined, or proceed with a guessed value.
Write a short rationale using fact slots and cite step IDs.	Write digits, spelled-out quantities, unit conversions, percentages, or a verdict.
Explain the audit trail in plain language with step references.	Introduce a cause, benchmark, or comparison that is not present in the trace.

4.2 Context assembly

The planner sees a bounded, metadata-only context. It never receives a ledger extract. The context has four parts:

- **Policy prompt.** Fixed, versioned, and reviewed. Defines the role, the prohibition on arithmetic, the schema, the clarification rule, and the treatment of untrusted text.
- **Capability card.** Dataset and snapshot name, available periods, entities, plants, products, configured currency and units, supported analyses, analysis readiness, and a one-line data-quality status. Example below.
- **Conversation summary.** The last six turns compressed to intent, resolved parameters, prior run IDs, and the facts that were shown. No raw rows.
- **Metadata results.** Output of lookup tools only: dimension candidates, account matches, quality gate summaries. Capped at 20 rows per lookup.

```
{
  "dataset": "Pilot manufacturer — Sep 2026 close",
  "snapshot_id": "snap_7f3c…",
  "periods": ["2026-07", "2026-08", "2026-09"],
  "entities": [{"id": "ENT-01", "name": "Northline Industries"}],
  "plants": [{"id": "PLT-01", "name": "Pune"}, {"id": "PLT-02", "name": "Coimbatore"}],
  "products": [{"id": "PRD-A", "name": "Standard motor"}, {"id": "PRD-B", "name": "Custom motor"}],
  "currency": "INR",
  "analysis_ready": {"variance": true, "decisions": true, "costing_bridge": true, "abc": false},
  "data_quality": {"tag_coverage_weighted": 0.997, "reconciled": true, "blocking_gates": 0},
  "supported_decisions": ["make_or_buy", "special_order", "add_or_drop", "sell_or_process",
                          "operate_or_shutdown", "product_mix", "cvp"],
  "limits": {"max_tool_calls": 3, "max_clarifications": 1}
}
```

The capability card is generated from the snapshot and the tool registry. It is not written by the model, and it must not claim an analysis is ready when a blocking gate is open.

4.3 Plan schema

The planner's entire output is a `Plan` object. There is no free-form tool-call syntax and no code execution. Money and quantities are strings to avoid floating-point drift; the orchestrator parses them into `Decimal` before execution.

```
{
  "intent": "special_order",
  "tool": "run_differential_analysis",
  "tool_version": "1.0.0",
  "arguments": {
    "decision_type": "special_order",
    "entity_id": "ENT-01",
    "plant_id": "PLT-01",
    "period_id": "2026-09",
    "product_id": "PRD-B",
    "order_units": "2000",
    "order_price": {"amount": "380.00", "currency": "INR"},
    "capacity_state": "idle",
    "incremental_fixed_cost": {"amount": "40000.00", "currency": "INR"},
    "variable_selling_cost_avoided": true
  },
  "assumptions": [
    {"key": "capacity_state", "value": "idle",
     "source": "user_implied", "evidence": "user said the plant has spare capacity",
     "confidence": 0.84, "confirm": true},
    {"key": "variable_selling_cost_avoided", "value": true,
     "source": "policy_default", "evidence": "special-order export treatment",
     "confidence": 0.70, "confirm": true}
  ],
  "missing": [],
  "clarification": null,
  "confidence": 0.88
}
```

Field	Validation	Rejection consequence
`intent`	Must be in the registered intent enum.	Return capability response; do not guess a nearby intent.
`tool`	Must be allowlisted and compatible with the pinned tool-registry version.	Plan rejected; logged as a planner defect.
`arguments`	Strict JSON Schema, `additionalProperties: false`; enums; decimal strings; currency and unit objects.	Schema error; one repair attempt, then clarification.
Arithmetic fields	Fields named `value`, `total`, `profit`, `variance`, `result` are forbidden in arguments.	Plan rejected as a trust-boundary violation.

Field	Validation	Rejection consequence
IDs	Must resolve exactly or uniquely through a metadata lookup.	Ambiguity returns candidates; no fuzzy execution.
`assumptions`	Every argument not supplied by the user must appear here with a source and confidence.	Missing assumption blocks execution.
`confidence`	Below 0.60 forces clarification even if the schema is valid.	No tool call.
`clarification`	At most one question, with 2–4 suggested answers and the fields it unblocks.	Multiple questions are merged by the orchestrator.

4.4 Tool catalogue

Tools are coarse enough to produce a complete working paper and narrow enough to validate. Each tool returns the standard envelope: result, facts, assumptions, warnings, remediation, trace reference, and snapshot ID. Phase 1 tools are frozen at M0; additions are versioned.

Tool	Purpose and key inputs	Returns	Milestone
`describe_dataset`	Capability card for a snapshot; no business inputs.	Periods, dimensions, readiness, quality summary.	M1
`resolve_dimension`	Type plus query string; entity, plant, product, period, account.	Ranked exact candidates with IDs; never a single fuzzy match without confidence 1.0.	M1
`search_accounts`	Name or code fragment, statement, behavior filter.	Up to 20 accounts with tag status.	M1
`get_data_quality`	Snapshot ID.	Gate results, blocking issues, coverage by amount.	M1
`run_variance_analysis`	Scope, period, basis, threshold profile, account or product filter.	Standard-cost decomposition or account flux with drill-down and residual.	M2
`run_flexible_budget`	Scope, period, actual volume basis.	Flexed budget, flexible and volume variances, line decomposition.	M2
`run_differential_analysis`	Decision type plus the specific parameter set; assumptions explicit.	Differential table, verdict, conditions, sensitivity, qualitative checklist.	M3
`run_costing_bridge`	Scope and period; inventory movement source.	Variable and absorption operating income, bridge, unreconciled residual.	M2
`explain_run`	Prior run ID.	Trace summary, rules, sources, assumptions, validator results.	M4
`run_abc_profitability`	Cost pools, drivers, activities, period, allocation method.	Traditional versus ABC product/customer profitability, unused capacity.	M7 (Phase 2)
`run_budget_chain`	Budget version, scenario overlay, driver assumptions.	Sales-to-cash budget chain, version diff, scenario comparison.	M9 (Phase 2)
`run_performance_eval`	Division scope, asset base, required return, scorecard weights.	ROI, residual income, controllable margin, scorecard contribution.	M10 (Phase 2)

4.5 Confirmation protocol

Confirmation is the moment the product earns or loses trust. The user must see what the system believes before it calculates, especially when the belief came from implication rather than explicit statement.

- **Every argument has a source:** `user_explicit`, `user_implied`, `registry`, `lookup`, `policy_default`, or `engine_default`.

- **Confirmation is mandatory** for `user_implied`, `policy_default`, and any `registry` or `lookup` value that changes a policy: capacity state, avoidable versus unavoidable cost, costing basis, allocation method, materiality threshold, or a what-if operator.
- **The parameter sheet** shows the argument label, resolved dimension name, value with unit, source badge, and an edit control. It also shows the fields the plan did not use, so the user can catch a missing driver.
- **Edits create a plan version.** The run pins the version; the audit record stores the diff between the model's proposal and the user's confirmed values.
- **Capacity conflicts are surfaced, not resolved.** If the user says "idle capacity" but the snapshot shows utilization above 95% for the period, the sheet warns with the utilization figure and requires an explicit override reason. The engine never silently substitutes an opportunity cost.
- **Auto-execution is narrow.** Only plans with all-explicit arguments, no policy-bearing fields, and complete data may run without a confirmation step. Even then, the analysis card repeats the parameter summary.

> **Do not let the model state a recommendation before confirmation.** "It looks profitable" in the planning turn is a verdict without a calculation. The planner is instructed to describe the proposed analysis, not its outcome. The UI renders the plan as "I will compare X against Y using Z assumptions", never as a conclusion.

4.6 Narration: slots, not sentences with numbers

The White Paper's rule is that the model narrates results. The implementation makes that literal. The response has three blocks, and only the middle one involves the model:

1. **Verdict block — deterministic.** The engine returns a decision enum (`accept`, `reject`, `investigate`, `favorable`, `adverse`, `mixed`). The renderer selects a fixed sentence template. The model cannot change it.
2. **Rationale block — model, slot-only.** The model writes two to four sentences using placeholders such as `{{f1|INR}}`. It receives the fact table but is forbidden from restating a value manually.
3. **Evidence block — deterministic.** Tables, waterfall, assumptions, source links, step list, and export. Always present, always formatted by code.

```
Accepting the order adds {{f1|INR}} of incremental profit. The floor price is
{{f2|INR}} per unit, and the offer leaves {{f3|INR}} per unit of headroom.
The conclusion holds only while the capacity assumption is confirmed; if it changes, rerun the analysis.
```

The renderer receives a fact table like this one and substitutes only the referenced values:

Fact	Label	Value	Unit	Step
`f1`	Incremental profit	120,000.00	INR	`s6`
`f2`	Floor price per unit	320.00	INR/unit	`s7`
`f3`	Headroom per unit	60.00	INR/unit	`s8`

Allowed placeholder grammar

`{{fact_id|format}}` only. Formats are a fixed allowlist: `INR`, `INR0`, `PCT1`, `PCT2`, `QTY0`, `QTY2`, `RATIO2`, `HOURS1`. Formatting is locale-aware and configured per tenant; Indian digit grouping is used for INR by default, international grouping otherwise. No arithmetic is permitted inside a placeholder. A model that wants to say "twice" must cite a ratio fact; a model that wants "about ₹320" must cite the exact fact and drop the hedge.

Validation of the rationale

Check	Rule	On failure
Unknown slot	Every placeholder must reference a fact in the run's fact table.	Reject; regenerate once.

Check	Rule	On failure
Raw numerals	After removing placeholders, no digit characters are allowed except inside allowed rule IDs and dates returned as facts.	Reject; regenerate once.
Spelled quantities	Number words adjacent to a currency or unit (“forty thousand rupees”, “two thousand units”) are rejected.	Reject; regenerate once.
Verdict fidelity	The rationale may not assert a decision; decision words are limited to the template block.	Reject; template fallback.
Polarity	If all cited net facts are negative, optimistic words (“gain”, “improves”, “headroom”) fail a soft check.	Regenerate with a polarity reminder; then fallback.
Length	Maximum 120 words.	Truncate at sentence boundary; log.
Citations	At least one step ID must be cited when the rationale makes a causal claim.	Reject; generic template rationale.

After a second failure, the renderer uses a pre-written rationale for the intent, populated with the same slots. The answer still ships, still auditable, and the validation failure is logged as a model-quality incident. The product degrades in prose quality, never in numerical integrity.

4.7 Failure modes and fallbacks

Condition	Behaviour	What the user sees
Low planner confidence	No tool call; one clarification with suggested interpretations.	“I can run this as a special-order analysis or a product-mix analysis. Which fits?”
Unsupported intent	Capability response listing supported analyses and the nearest supported framing.	“I can’t build a channel-mix model yet. I can compare contribution by product under a capacity constraint.”
Missing mandatory data	Stop; remediation card with the exact missing fields and upload template.	“Actual material quantities are missing for PLT-02. Upload the usage file or connect the subledger.”
Blocking quality gate	Stop; list failed gates, affected accounts by amount, and links to fix tags.	“2.4% of cost is untagged. Tag these six accounts to continue.”
Ambiguous dimension	Return candidates with IDs; no fuzzy execution.	“Two products match ‘custom motor’. Select one.”
Capacity conflict	Warn; require override with reason before execution.	“You said idle capacity, but September utilization was 98%. Override or change the assumption.”
Engine domain error	Structured error with the violated pre-condition and a fix.	“Contribution per unit is negative; a break-even volume does not exist. Check price and variable cost.”
Provenance validation failure	Template fallback; incident logged with the raw rationale.	Complete answer with template prose and a trace; no mention of an internal validator.
Model timeout or outage	Retry once; then switch to workflow lane and tables-only response.	“Narrative is unavailable. The analysis below is complete and sourced.”
Prompt injection in data	Text treated as data; no tool call, no policy change; incident flagged.	Normal answer; the malicious string is visible only in the source record.

4.8 Prompt and model governance

- Prompts live in the repository as versioned files. Their hash is stored in every run. A prompt change requires the planner eval subset to pass and a domain review.

- Model ID, provider, temperature (0), top-p (1), and token limits are pinned per run. A fallback model is allowed only if it passes the same eval suite; the fallback is recorded.
- No customer data appears in prompt templates or few-shot examples. Examples use synthetic accounts and amounts.
- Tool descriptions are treated as code: reviewed, tested for ambiguity, and versioned with the tool.
- A red-team suite runs on every prompt or model change: instruction override, invented account, “just estimate it”, arithmetic request, unit conversion, and a ledger row containing an instruction.

5 Cost data layer

The data layer is where projects of this kind fail. The intelligence is worthless on untagged data, so the registry and the gates are the first deliverable, not an afterthought. Two rules govern the design: every fact is dimensional and every classification is versioned. A third rule governs behaviour: the system would rather refuse an analysis than silently infer a fixed/variable split.

5.1 Principles

- **Registry first.** No analysis runs against an account whose behaviour and direct/indirect status are unknown, unless the analysis does not depend on that classification and the omission is disclosed.
- **One canonical fact shape.** Amounts, quantities, prices, hours, and volumes are separate typed facts, not strings in a spreadsheet column.
- **Immutable snapshots.** Ingestion creates a new snapshot; it never edits a prior one. Runs pin a snapshot.
- **No silent coercion.** Unparseable rows fail the load with row-level errors. Unknown dimension members go to a mapping queue.
- **Weighted quality.** Coverage is measured by cost amount, not by account count. A single untagged account holding 8% of cost is a blocking problem even if 98% of accounts are tagged.

5.2 Dimensional model

Dimension	Grain	Key attributes	Notes
`dim_entity`	Legal entity	ID, name, currency, country, parent	Phase 1 supports one functional currency per entity; no translation.
`dim_plant`	Manufacturing or service location	ID, entity, name, capacity, calendar	Capacity used for utilization and volume-variance checks.
`dim_product`	Product or product family	ID, name, family, UOM, standard-cost version	Customer dimension arrives in Phase 2.
`dim_period`	Fiscal month	ID, year, period number, start, end, status	Facts are period-keyed; timestamps are metadata only.
`dim_account`	Account × plant (optional override)	Code, name, statement, type, behavior, direct/indirect, controllability	Base classification plus plant-specific overrides; SCD2 versioned.
`dim_cost_center`	Responsibility center	ID, plant, manager, controllability scope	Used for accountability and exception routing.
`dim_activity` (P2)	Activity	ID, pool, driver, capacity	ABC and TDABC.
`dim_customer` (P2)	Customer or segment	ID, segment, channel, region	Customer profitability.

5.3 Core tables

Table	Key columns	Used by
`fact_gl_actual`	`snapshot_id, period_id, entity_id, plant_id, account_id, cost_center_id, amount, currency, source_doc_id, load_id`	Flux, registry coverage, budget-vs-actual, bridges.
`fact_standard_cost`	`period_id, product_id, plant_id, version, material_cost, labor_cost, variable_oh, fixed_oh, unit_cost, uom`	Standard-cost variances, absorption costing, CVP.
`fact_standard_usage`	`period_id, product_id, plant_id, material_qty_per_unit, labor_hours_per_unit, machine_hours_per_unit, uom`	SQ and SH calculation; flexible budget.
`fact_actual_usage`	`period_id, product_id, plant_id, material_qty_purchased, material_qty_used, actual_price, actual_labor_hours, actual_rate, actual_machine_hours`	Price/usage/rate/efficiency decomposition.
`fact_volume`	`period_id, product_id, plant_id, produced_units, sold_units, beg_fg_units, end_fg_units, beg_rm_units, end_rm_units`	Volume variances, flexible budget, inventory bridge.
`fact_budget`	`period_id, version, entity_id, plant_id, account_id, amount, volume_basis, currency`	Flux, flexible budget, monitoring.
`fact_inventory_balance`	`period_id, product_id, plant_id, units, standard_cost, fixed_oh_rate`	Costing-method bridge, absorption profit.
`fact_actual_price_volume`	`period_id, product_id, plant_id, actual_price, actual_units, actual_variable_cost_per_unit`	Sales and contribution variances.
`registry_account_version`	`account_id, plant_id, version, behavior, behavior_method, direct_indirect, controllability, approved_by, valid_from, valid_to`	Every classification-dependent analysis.
`snapshot`	`snapshot_id, tenant_id, created_at, manifest_hash, registry_version, schema_version, row_counts, status`	Pinning and re-performance.
`load` , `load_error`	`load_id, snapshot_id, file, gate_results, error_count, status`	Ingestion audit.
`run` , `run_step` , `audit_event`	See Section 7.	Answer trace and re-performance.
`account_mapping` (P2)	`source_system, source_account, target_account, status, approved_by, contract_version`	ERP connectors.

5.4 Account registry

The registry is a versioned classification of every account, with optional plant-level overrides. It answers four questions the engine cannot proceed without: is the account fixed, variable, or semi-variable; is it direct or indirect; is it controllable by the manager being evaluated; and is it manufacturing or period cost.

Behaviour	Definition	Engine use	Estimation method
Fixed	Does not change with volume inside the relevant range.	Period cost in contribution margin; spending variance; shutdown analysis.	User classification; contract or budget evidence.
Variable	Changes proportionally with volume or driver.	Contribution margin; CVP; differential cost; flexible budget.	Standard cost card; user classification; high-low check.

Behaviour	Definition	Engine use	Estimation method
Semi-variable	Fixed base plus variable component.	CVP, flexible budget, break-even; split stored explicitly.	User-specified split or high-low over at least six periods; regression in Phase 2.
Step-fixed (P2)	Fixed within a capacity band, jumps at thresholds.	Capacity planning, relevant range warnings.	Band table supplied by the client.

Relationship to standard cost cards. The registry classifies GL accounts and period costs. Standard cost cards carry the per-unit material, labour, and overhead standards used for product costing and standard-cost variances. The engine must not add both: product-level variable cost comes from the standard cost card; period fixed cost comes from the registry and GL. Where a card is missing, the engine blocks the product-level analysis and names the missing card; it does not fall back to account averages.

Registry lifecycle

1. **Import** the chart of accounts with statement, type, and cost-center mapping.
2. **Suggest** behaviour and direct/indirect using deterministic rules first (account type, name patterns, standard-cost mapping), then a model suggestion as a secondary hint.
3. **Review** in a bulk grid sorted by amount, so the analyst spends time on material accounts. Every suggestion shows its evidence.
4. **Approve** with a named approver and timestamp. Approval is per version, not per row.
5. **Version** any change as a new registry version with a diff and reason. Runs pin the version they used.

Weighted coverage is computed as tagged cost amount divided by total cost amount for the analysis scope. The pilot target is 99.9% or better; below 99% is blocking, and any single untagged account above 2% of scope is blocking regardless of the total.

5.5 Ingestion pipeline

1. **Receive.** Upload or connector extract; allowlist XLSX and CSV; 100 MB per file; macro-free; virus scan; zip-bomb protection.
2. **Stage.** Load raw values as strings into staging tables with file, sheet, row, and column references.
3. **Map.** Resolve columns to the canonical template; unresolved required columns fail the load with a mapping error.
4. **Type.** Parse amounts, quantities, prices, hours, dates, and periods with explicit formats. No locale guessing beyond the declared tenant locale.
5. **Resolve.** Match dimension members exactly; unknowns go to a mapping queue and the load reports them.
6. **Validate registry.** Every cost account in scope must have a behaviour and direct/indirect tag, or an approved override.
7. **Run gates.** G1–G12 below. Blocking failures stop the canonical load.
8. **Reconcile.** Compare loaded totals to uploaded control totals and to the prior period where applicable.
9. **Load canonically.** Write typed facts in one transaction per load.
10. **Freeze.** Build Parquet files, compute hashes, write the manifest, and register the snapshot.
11. **Report.** Show a quality report with pass/warn/block per gate, affected amounts, and remediation links.

5.6 Quality gates

Gate	Check	Severity	Remediation
G1	File and column schema matches the template version.	BLOCK	Download the current template; remap columns.
G2	Referential integrity: every fact resolves to a dimension member.	BLOCK	Map unknown members; add missing dimensions.

Gate	Check	Severity	Remediation
G3	Period completeness: all expected periods and plants present.	BLOCK	Upload the missing period or exclude the plant explicitly.
G4	No duplicate load for the same file hash and period.	BLOCK	Use the existing snapshot or load a corrected file with a new version.
G5	Control-total reconciliation within ₹1 or 0.01% of the control total, whichever is larger; the tolerance is configurable per contract.	BLOCK	Review rejected rows and control totals; re-extract.
G6	Weighted account-tag coverage at or above 99%; no single untagged account above 2% of scope.	BLOCK	Tag accounts in the registry; approve overrides.
G7	Unit of measure present and consistent for every quantity, price, and rate.	BLOCK	Supply UOM; fix conflicting conversions.
G8	Single functional currency per entity; no mixed-currency facts in scope.	BLOCK	Filter to one currency or split the entity; translation is out of scope.
G9	Sign convention: costs positive, credits negative, revenue positive; reversals flagged.	WARN	Review the sign report; confirm or correct.
G10	Outliers: period-over-period change above 3× or z-score above 4.	WARN	Investigate before relying on the variance; never block solely on an outlier.
G11	Standard cost card present for every product in scope, with all components.	BLOCK	Load the missing card; exclude the product with disclosure.
G12	Standard usage and actual usage present for the products being decomposed.	BLOCK	Load usage data or run the total-variance-only view with disclosure.

5.7 Snapshot semantics

```
{
  "snapshot_id": "snap_7f3c9a…",
  "tenant_id": "tnt_pilot_01",
  "created_at": "2026-11-02T09:14:00Z",
  "registry_version": 7,
  "schema_version": "1.3.0",
  "transform_version": "1.1.0",
  "files": [
    {"name": "fact_gl_actual.parquet", "sha256": "…", "rows": 184203},
    {"name": "fact_standard_cost.parquet", "sha256": "…", "rows": 640}
  ],
  "control_totals": {"gl_cost_inr": "48210344.00", "loaded_inr": "48210344.00"},
  "quality": {"blocking": 0, "warnings": 2, "coverage_weighted": 0.9974}
}
```

The manifest hash is the snapshot ID's source of truth. Runs pin the snapshot ID and store the manifest hash. Re-performance verifies both. A new ingestion always creates a new snapshot; corrections are new snapshots with a lineage note, never in-place edits.

5.8 The variance input problem

Most GL extracts carry account amounts, not the actual quantities, prices, and hours needed for a price/usage decomposition. The product handles this in three explicit levels rather than estimating:

Level	Inputs available	Output and disclosure
A. Standard-cost decomposition	Standards, actual quantities/price/hours, actual output.	Full MPV/MUV, LRV/LEV, overhead spending/efficiency. No disclosure beyond basis.

Level	Inputs available	Output and disclosure
B. Account flux	GL actuals, budget, prior period.	Budget-vs-actual and period-over-period movement with drill-down. No fabricated price/usage split; residual labelled “unexplained”.
C. Flexible budget	GL actuals, budget, actual volume, variable-cost standards.	Volume effect isolated from spending/price/efficiency effects. Mix variance only when product mix data is available.

If a user asks for a decomposition at level B, the answer states which components cannot be computed and offers the upload path. The engine may still compute the total variance; it must label the decomposition unavailable. This is the difference between an honest partial answer and a confident fiction.

5.9 Phase 2 schema extensions

The Phase 1 schema leaves room for the Phase 2 modules without renaming core facts. The additions are additive tables plus two new dimension types.

Table	Purpose	Module
`dim_activity`, `fact_cost_pool`	Activity definitions, pool costs, capacity, and period.	ABC / TDABC (M7)
`dim_driver`, `fact_driver_event`	Driver definitions and event-level consumption (setups, inspections, orders, movements).	ABC / TDABC (M7)
`fact_allocation`	Pool-to-product/customer allocation with method, rate, driver quantity, and unused capacity.	ABC (M7)
`dim_customer`, `fact_customer_activity`	Customer-level driver consumption and profitability.	ABC (M7)
`budget_version`, `budget_node`, `scenario_overlay`	Versioned budget chain and scenario diffs.	Budgeting (M9)
`kpi_definition`, `scorecard`, `scorecard_result`	Metric definitions, weights, and computed results.	Performance (M10)
`alert_rule`, `alert_instance`	Natural-language rule compiled to engine queries; ranked alerts.	Monitoring (M8)
`connector_contract`, `mapping_version`	Source system, control totals, approved account mappings, contract version.	Connectors (M6)

6 Deterministic engine

The engine is the authority. It is a library of pure functions with no database access, no network calls, and no model calls. It receives typed inputs, applies a registered rule, and returns a value plus the exact steps that produced it. If the engine cannot compute something under the stated policy, it raises a domain error with a remediation code. It never fills a gap with an estimate.

6.1 Calculation result object

```
@dataclass(frozen=True)
class CalcStep:
    step_id: str                  # s1, s2, … stable within a run
    label: str                    # "Incremental revenue"
    rule_id: str                  # "DA-SO-001"
    rule_version: str             # "1.0.0"
    formula: str                  # "order_units × order_price"
    inputs: dict[str, Decimal | str]
    value: Decimal
    unit: str                     # "INR", "INR/unit", "units", "hours"
    source_refs: list[str]        # fact row IDs or aggregate query IDs
    note: str | None = None

@dataclass(frozen=True)
class CalcResult:
    value: Decimal
    unit: str
    favorable: bool | None        # true = favorable, false = adverse, none = neutral
    verdict: str | None           # accept / reject / investigate
    steps: list[CalcStep]
    facts: list[Fact]
    assumptions: list[AssumptionRef]
    warnings: list[Warning]
    sensitivity: list[SensitivityPoint]
```

`Fact` is the render-facing projection of a step: a stable ID, a human label, a formatted value, a unit, and the step that produced it. The renderer never derives a fact; it only formats one.

6.2 Module layout

```
packages/engine/costpilot_engine/
  money.py            Decimal construction, currency, rounding policy
  units.py            UOM registry, explicit conversions, dimension checks
  trace.py            CalcStep, CalcResult, Fact, assumption and warning models
  rules/
    registry.py       rule registration, versions, source citations
    catalog.py        human-readable rulebook generated from the registry
  variance/
    material.py       MPV, MUV, MCV, purchase-basis supplement
    labor.py          LRV, LEV, LCV
    overhead.py       variable spending/efficiency; fixed spending/volume
    flexible_budget.py
    flux.py           account-level budget/prior-period movement
  decisions/
    differential.py   relevant-cost primitives
    make_or_buy.py
    special_order.py
    add_or_drop.py
    sell_or_process.py
    shutdown.py
    product_mix.py
    cvp.py
  costing/
    variable.py       contribution-format income statement
    absorption.py     gross-profit-format income statement
    bridge.py         fixed-overhead-in-inventory reconciliation
  abc/                Phase 2
    pools.py  drivers.py  traditional.py  tdabc.py  profitability.py
  budgets/            Phase 2
    chain.py  cash.py  scenarios.py
  performance/        Phase 2
    roi.py  residual_income.py  scorecard.py
  validation/
    identities.py  units.py  coverage.py
```

6.3 Numeric, unit, and rounding policy

- **Decimal only.** Construct from strings. Floats are banned in the engine package by lint. Division uses an explicit context with at least 28 digits.
- **No rounding mid-calculation.** Steps carry full precision. Rounding happens once, at rendering: currency 2 decimals, rates 4, quantities 4, percentages 2. A displayed total may differ from the sum of displayed components by ₹0.01; the trace shows a rounding-adjustment line and the full-precision values.
- **Units are dimensional.** Amounts are currency; quantities are UOM; rates are currency per UOM. The engine refuses to add incompatible units. Conversions happen only through the explicit UOM registry, never implicitly.
- **Currency is per entity.** Phase 1 rejects mixed-currency scopes. There is no FX translation and no display-currency conversion.
- **Sign convention.** Internally, a variance is positive when favorable. API results carry a signed value and a `favorable` boolean. The UI prints the magnitude with an F/A label, never a negative number for a variance. Actual losses use parentheses.
- **Deterministic order.** Steps are ordered by dependency, then by rule ID. No unordered iteration or locale-dependent sorting affects numbers.

6.4 Variance rulebook

Symbols: AQ = actual quantity, AP = actual price, SQ = standard quantity allowed for actual output, SP = standard price, AH = actual hours, AR = actual rate, SH = standard hours allowed for actual output. Every formula below produces a favorable-positive value; the display shows magnitude plus F/A.

Variance	Formula	Favorable when	Note
Material price (MPV)	$AQ_{used} \times (SP - AP)$	AP < SP	Usage basis for tie-out to MCV. Purchase-basis price variance reported separately when purchased ≠ used.
Material usage (MUV)	$SP \times (SQ - AQ_{used})$	AQ < SQ	SQ = actual output × standard qty per unit.
Material cost (MCV)	`MPV + MUV`	—	Identity checked to ₹0.01.
Labour rate (LRV)	`AH × (SR − AR)`	AR < SR	Overtime premium policy is explicit in the rule.
Labour efficiency (LEV)	`SR × (SH − AH)`	AH < SH	SH = actual output × standard hours per unit.
Labour cost (LCV)	`LRV + LEV`	—	Identity checked.
Variable overhead spending	$Actual\ VOH - (AH \times SR_v)$	Actual < applied	Rate is the variable overhead rate per hour.
Variable overhead efficiency	$SR_v \times (AH - SH)$	AH < SH	Only meaningful when overhead is driven by hours.
Fixed overhead spending	`Budgeted FOH − Actual FOH`	Actual < budget	Controllable in the short run.
Fixed overhead volume	`Applied FOH − Budgeted FOH`, where applied = $SR_f \times SH$	Applied > budget	Capacity utilization; not a spending failure.
Total overhead	`Applied − Actual`	Applied > actual	Equals spending + volume for fixed; spending + efficiency for variable.
Sales price	`Actual revenue − (Actual units × Budget price)`	Actual price > budget	Revenue sign convention; positive is favorable.
Sales volume	`(Actual units − Budget units) × Budget CM per unit`	Actual > budget	Contribution basis, not revenue basis.

Flexible-budget restatement

The flexible budget is the default comparison for performance. It restates revenue and variable costs at actual volume, leaves fixed costs at budget, and separates the volume effect from the spending effect. The identity is **static-budget variance = flexible-budget variance + sales-volume variance**.

Line	Static budget	Flexed at 11,000	Actual	Flexible variance
Units	10,000	11,000	11,000	—
Revenue at ₹500 / ₹490	5,000,000	5,500,000	5,390,000	**110,000 A**
Variable cost at ₹320 / ₹325	3,200,000	3,520,000	3,575,000	**55,000 A**
Contribution margin	1,800,000	1,980,000	1,815,000	**165,000 A**
Fixed cost	900,000	900,000	920,000	**20,000 A**
Operating income	**900,000**	**1,080,000**	**895,000**	**185,000 A**

The static-budget variance is ₹5,000 adverse. The sales-volume variance is ₹180,000 favorable (1,000 units × ₹180 budgeted contribution). The flexible-budget variance is ₹185,000 adverse, decomposed into price ₹110,000, variable cost ₹55,000, and fixed spending ₹20,000. Without the flex, a manager would be blamed for a ₹5,000 miss while the underlying spending deterioration is 37 times larger.

Material decomposition example

Standard: 2 kg per unit at ₹50. Actual output: 10,000 units. Actual usage: 20,500 kg at ₹52. MPV = 20,500 × (50 – 52) = ₹41,000 adverse. MUV = 50 × (20,000 – 20,500) = ₹25,000 adverse. MCV = ₹66,000 adverse. If 21,000 kg were purchased, a purchase-basis price variance of ₹42,000 adverse is reported separately, with the ₹1,000 difference held in the 500 kg of inventory at standard; the usage-basis MCV remains unchanged.

6.5 Short-run decision rulebook

Every workflow applies the same relevant-cost primitives: include future costs that differ between alternatives; exclude sunk costs; exclude allocated common fixed costs unless they are avoidable; include opportunity cost when a scarce resource is redeployed. The engine produces a differential table, a verdict, conditions, and a qualitative checklist. The checklist does not change the numbers; it can change the verdict from unconditional to conditional.

Decision	Core comparison	Must include / exclude	Guardrail
Make or buy	Avoidable cost of making versus purchase cost plus incremental fixed cost.	Include avoidable fixed, opportunity cost of freed capacity. Exclude allocated common fixed and sunk tooling.	Flag quality, supply continuity, and capacity-release assumptions. Verdict conditional if any unresolved.
Special order	Incremental revenue versus incremental variable cost, incremental fixed, and lost contribution.	Include opportunity cost if capacity is constrained. Exclude existing fixed overhead allocation.	Confirm idle capacity against utilization; flag cannibalization and price precedent.
Add or drop	Contribution margin lost versus avoidable fixed costs saved.	Include avoidable fixed savings and complementary-product effects. Exclude allocated fixed.	Explicit warning when a loss-reporting product still covers avoidable costs.
Sell or process further	Incremental revenue after further processing versus incremental processing cost.	Treat joint costs as sunk. Include incremental variable and avoidable fixed.	Flag capacity and quality constraints; no allocation of joint cost.
Operate or shut down	Contribution forgone versus avoidable fixed costs saved over the shutdown window.	Include avoidable fixed, restart costs where quantifiable. Exclude unavoidable fixed.	Window must be stated; flag workforce and customer-contract effects as qualitative.
Product mix	Rank by contribution per unit of the scarce resource.	Include the true limiting factor. Exclude fixed costs from ranking.	Single constraint only in Phase 1; flag if multiple constraints or integer indivisibilities bind.

Decision	Core comparison	Must include / exclude	Guardrail
CVP / break-even	Fixed cost divided by contribution per unit; margin of safety; target profit.	Use contribution format; state relevant range and constant-mix assumption.	No break-even when contribution per unit is zero or negative; flag mixed-product mix changes.

6.6 CVP and the costing-method bridge

Measure	Formula	Policy
Contribution per unit	`Price − Variable cost per unit`	Variable cost includes variable selling if applicable.
Contribution margin ratio	`Contribution ÷ Revenue`	Use unrounded contribution and revenue.
Break-even units	`Fixed cost ÷ Contribution per unit`	Undefined when contribution ≤ 0; raise a domain error.
Break-even revenue	`Fixed cost ÷ Contribution margin ratio`	Single-product or constant-mix only.
Target-profit units	`(Fixed cost + Target operating income) ÷ Contribution per unit`	Target is pre-tax operating income.
Margin of safety	`(Actual or budget sales − Break-even sales) ÷ Actual or budget sales`	State the base used.
Degree of operating leverage	`Contribution ÷ Operating income`	Undefined at zero operating income; warn.
Absorption–variable profit difference	`Change in inventory units × Fixed overhead rate per unit`	Positive when inventory builds; reconciliation must tie to ₹0.01.
ROI	`Operating income ÷ Average operating assets`	Average of beginning and ending assets unless the client supplies a method.
Residual income	`Operating income − (Required return × Average operating assets)`	Required return is a client input, not a default.

6.7 Worked example: the special order

The canonical demo: 2,000 units at ₹380, variable manufacturing cost ₹300 per unit, incremental fixed cost ₹40,000, idle capacity, no cannibalization, no variable selling cost on the order. Regular price is ₹500 and regular variable selling cost is ₹20 per unit.

Differential line	Per unit	2,000 units	Treatment
Incremental revenue	380.00	760,000.00	Relevant
Variable manufacturing cost	(300.00)	(600,000.00)	Relevant
Incremental contribution	**80.00**	**160,000.00**	—
Incremental fixed cost (setup)	(20.00)	(40,000.00)	Relevant
Opportunity cost of displaced sales	—	0.00	Idle capacity confirmed
Allocated existing fixed overhead	—	excluded	Sunk / unavoidable
Incremental profit	**60.00**	**120,000.00**	—
Minimum acceptable price	**320.00**	**640,000.00**	Floor

Verdict: accept, conditionally. The conditions are: idle capacity is real for the order window; no regular sales are displaced; the price does not become a precedent for regular customers; and the order passes quality, regulatory, and working-capital checks. If idle capacity were only 1,000 units, 1,000 regular units would be displaced, the opportunity cost would be ₹180,000 (1,000 × ₹180 regular contribution), the incremental profit would turn to ₹60,000 negative, and the floor would rise to ₹410. The engine computes this sensitivity automatically and shows it as a second row on the card.

6.8 Phase 2 formulas

ABC and TDABC

Pool rate = total pool cost ÷ total driver quantity. Allocated cost = pool rate × driver quantity consumed. TDABC capacity cost rate = cost of capacity supplied ÷ practical capacity; cost per transaction = rate × time per transaction; unused capacity cost = rate × (capacity − time used). Allocations must sum to the pool total within ₹0.01; unused capacity is shown as a separate line, never spread silently into products.

Example: a setup pool of ₹600,000 with 1,500 setups gives ₹400 per setup. A high-volume product consuming 200 setups receives ₹80,000; a low-volume custom product consuming 1,300 receives ₹520,000. A traditional machine-hour allocation that gives the high-volume product 80% of the pool would assign ₹480,000 and ₹120,000 — reversing the economics. The report always shows traditional and ABC side by side; the difference is the finding, not a hidden adjustment.

Budget chain

Production units = sales units + desired ending finished goods − beginning finished goods. Material purchases = production usage + desired ending raw material − beginning raw material. Labour hours = production units × standard hours per unit. Each node is a pure function with a versioned output; a scenario is an overlay diff, not a copied workbook. Example: sales 10,000 + ending 1,200 − beginning 1,000 = 10,200 units produced; at 2 kg per unit, usage is 20,400 kg; with ending 1,500 and beginning 1,200, purchases are 20,700 kg.

Performance measurement

ROI = operating income ÷ average operating assets; residual income = operating income − required return × average operating assets. Example: operating income ₹1,200,000 on assets ₹6,000,000 is 20% ROI. A new investment adds ₹180,000 of income on ₹1,200,000 of assets — 15%, above the 12% required return but below the division's current 20%. The manager rejects it under ROI; residual income rises by ₹36,000. The engine shows both views on the same data and labels the conflict explicitly, without declaring which manager behaviour is "right".

6.9 Edge cases and policy decisions

Case	Engine behaviour	User-facing consequence
Zero or negative volume	Domain error for per-unit and break-even rules; total-cost views may still run.	Explain the precondition and offer a total view.
Contribution ≤ 0	Break-even undefined; shutdown analysis still available.	"No break-even volume exists at this price."
Missing standard cost card	Block product-level variance and CVP; name the product and card.	Upload the card or exclude the product explicitly.
Purchase quantity ≠ usage	Usage-basis MPV for tie-out; purchase-basis price variance as a supplement; inventory effect shown.	Two clearly labelled price variances, not one ambiguous number.
Rounding residual	Full precision internally; display rounding adjustment when components do not sum exactly.	Tables tie; the adjustment is visible.
Mixed currency scope	Block; no FX translation in Phase 1.	Filter to one entity/currency or defer.
Unit mismatch	Block; require explicit conversion through the UOM registry.	Fix the input or the UOM mapping.

Case	Engine behaviour	User-facing consequence
Credits and reversals	Preserve sign; flag reversal pairs; do not net silently across periods.	Sign report on ingestion; drill-down shows the pair.
Volume outside relevant range	Compute but warn; do not extrapolate fixed costs or step costs.	Warning chip with the observed range.
Multiple scarce resources	Ranking is invalid; raise a domain warning and suggest linear programming in Phase 3.	“Two constraints bind; contribution-per-unit ranking is not sufficient.”
Joint costs	Sunk for sell-or-process-further; excluded from the incremental comparison.	Joint cost shown as memo only.
Idle capacity claim	Cross-check against utilization; require override if above 95%.	Assumption warning with evidence.
Cannibalization	No automatic estimate; user supplies displaced units or confirms none.	Conditional verdict until confirmed.
Depreciation and non-cash costs	Excluded from short-run cash differential unless the user explicitly includes a cash-equivalent charge.	Policy note on the differential table.

7 Audit and explainability

The audit trail is not a log file appended after the fact. It is the product’s evidence layer, built at the same time as the calculation and sealed before the answer is shown. A controller should be able to take any response, open the trace, and re-perform the calculation by hand or with the re-performance tool. If that takes more than a few minutes, the audit design has failed.

7.1 The run record

Field group	Contents
Identity	Run ID, tenant, user, session, origin (user, monitor, re-performance), parent run, timestamps.
Question and plan	Original question, normalized question, intent, tool and version, plan version, planner confidence, raw planner output, clarification asked.
Confirmation	Whether confirmation was required, confirmed by, timestamp, model-proposed values, user edits, and reasons for overrides.
Pins	Snapshot ID and manifest hash, registry version, engine semver, rulebook version, prompt version, model ID and provider, tool-registry version.
Inputs	Resolved arguments, dimension labels, assumptions with source and confirmation, data-quality gate results at run time.
Calculation	Every step: rule ID and version, formula, inputs, value, unit, source refs, warnings, sensitivity points.
Facts and response	Fact table, verdict, deterministic template version, raw rationale with placeholders, rendered rationale, table payloads, export references.
Validation	Schema, identity, unit, basis, coverage, assumption, provenance, and numeric-claim results; warnings; fallback used.
Integrity	Input hash, output hash, previous event hash, event hash, sealing timestamp.
Operations	Latency by stage, token counts, model cost, cache hits, retries, error codes.

7.2 Trace example


```
{
  "run_id": "run_01J9ZK4T7Q",
  "origin": "user",
  "question": "Should we accept 2,000 units at ₹380 with idle capacity?",
  "intent": "special_order",
  "tool": {"name": "run_differential_analysis", "version": "1.0.0"},
  "plan_version": 3,
  "confirmation": {
    "required": true, "confirmed_by": "u_1042", "at": "2026-11-02T10:22:41Z",
    "edits": [{"key": "capacity_state", "proposed": "idle", "confirmed": "idle",
               "reason": "order fits within documented spare shift"}]
  },
  "pins": {
    "snapshot_id": "snap_7f3c9a", "manifest_hash": "sha256:…",
    "registry_version": 7, "engine_version": "1.4.2",
    "rulebook_version": "2026.11.1", "prompt_version": "planner-1.6.0",
    "model": "provider/model@2026-10"
  },
  "assumptions": [
    {"id": "a1", "text": "Idle capacity exists for the order window",
     "source": "user_implied", "confirmed": true},
    {"id": "a2", "text": "No regular sales are displaced",
     "source": "user_explicit", "confirmed": true}
  ],
  "steps": [
    {"step_id": "s1", "rule_id": "DA-SO-001", "label": "Incremental revenue",
     "formula": "order_units × order_price", "value": "760000.00", "unit": "INR"},
    {"step_id": "s2", "rule_id": "DA-SO-002", "label": "Incremental variable cost",
     "formula": "order_units × variable_cost_per_unit", "value": "-600000.00", "unit": "INR"},
    {"step_id": "s3", "rule_id": "DA-SO-003", "label": "Incremental contribution",
     "formula": "s1 + s2", "value": "160000.00", "unit": "INR"},
    {"step_id": "s4", "rule_id": "DA-SO-004", "label": "Incremental fixed cost",
     "formula": "incremental_fixed", "value": "-40000.00", "unit": "INR"},
    {"step_id": "s5", "rule_id": "DA-SO-005", "label": "Opportunity cost",
     "formula": "displaced_units × regular_contribution_per_unit", "value": "0.00", "unit": "INR"},
    {"step_id": "s6", "rule_id": "DA-SO-006", "label": "Incremental profit",
     "formula": "s3 + s4 + s5", "value": "120000.00", "unit": "INR"}
  ],
  "facts": [{"fact_id": "f1", "step_id": "s6", "value": "120000.00", "unit": "INR"}],
  "validation": {"schema": "pass", "identities": "pass", "units": "pass",
                 "coverage": "pass", "provenance": "pass"},
  "response_hash": "sha256:…",
  "latency_ms": {"plan": 1240, "execute": 210, "render": 1680, "total": 3130}
}
```


7.3 Hash chain and immutability

- Events are canonicalized as JSON with sorted keys, no insignificant whitespace, UTF-8, and decimals as strings.
- `event_hash = SHA256(previous_hash || canonical_json(event))`. The run seal stores the final hash. Any later mutation breaks the chain.
- Audit rows are append-only in Postgres; a trigger rejects updates and deletes. Sealed run exports are written to object storage with object lock.
- A verification endpoint recomputes the chain for a run or a date range and reports the first broken link.
- Retention: audit metadata and traces for seven years; raw snapshots for thirteen months in the pilot, configurable per contract. Deleting a snapshot does not delete its audit metadata.

7.4 Re-performance

Re-performance is the operational definition of trust. The user opens a run, selects “Re-perform”, and the system re-executes the pinned tool against the pinned snapshot with the confirmed arguments. Every step value is compared to the stored value. The result is one of three states: **exact match**, **display-rounding difference** (documented, within the display tolerance), or **mismatch** (a defect, with the differing steps shown).

- The engine wheel and container image for each released version are retained so a re-performance uses the same code, not today's code.
- The comparison is numeric only. Prose may differ because a model version changed; that is not a numerical mismatch.
- A command-line re-performance tool supports sampled audits: `costpilot reperform ––run run_01J9ZK4T7Q ––json`.
- Every completed run is sampled for nightly re-performance. A mismatch rate above zero is a release-blocking incident.

7.5 Analysis-card requirements

The card is the product surface. It must satisfy a controller who wants to check the answer and an analyst who wants to act on it. The following are mandatory, not stylistic:

Element	Requirement	Why
Header	Restated question, period, entity, plant, product, basis, snapshot badge, run status.	Prevents answering the wrong scope or period.
Verdict	Engine-selected decision label and one fixed sentence. No hedging, no model-written conclusion.	Makes the recommendation auditable and stable.
KPI strip	Three to five headline facts, each linked to its step.	Fast scan without losing provenance.
Differential table	Every relevant line, relevant/excluded treatment, per-unit and total columns, F/A labels.	The working paper the analyst would otherwise build.
Waterfall	Variance decomposition with the residual labelled; no chart-derived numbers.	Pattern recognition without arithmetic in the browser.
Assumptions	Chips with source, confirmation state, edit affordance, and the effect of changing the assumption.	Prevents silent policy and capacity assumptions.
Audit drawer	Step list, rule IDs, formula, inputs, source records, validation results, re-perform action.	Re-performance path.
Actions	Export XLSX/PDF/JSON, re-perform, ask follow-up, open data-quality report.	Moves the answer into the close process.
Remediation state	When blocked, the card becomes an actionable remediation panel, not a generic error.	No dead ends.

Formatting rules. Finance tables are right-aligned with tabular figures. Variance magnitudes carry F/A text, not colour alone. Negative actual amounts use parentheses. Currency and date grouping follow the tenant locale. A number on screen must be present in the response payload; the client never derives a subtotal, percentage, or difference for display.

7.6 Claim mapping

Each rationale sentence maps to the facts it cites. The UI can highlight a sentence and show the corresponding step and source records. Qualitative claims map to an assumption or a rule citation; they cannot carry an implicit number. Suggested follow-up questions are selected from deterministic templates based on the tool, verdict, and capability card, so they cannot introduce a new analysis that the system cannot run.

This mapping is also the basis for the "what would change this" section: the engine returns sensitivity points (for example, the floor price at different capacity states), and the renderer links each to the assumption that controls it. The model may phrase the link, but the values and the controlling assumption come from the trace.

8 API and interface contracts

The API is deliberately small. A session creates a plan; a plan is confirmed; a run executes and streams stages; a completed run exposes results, trace, re-performance, and exports. There is no endpoint that accepts free-form SQL or arbitrary computation. Every response is typed and versioned.

8.1 Endpoints

Endpoint	Purpose	Notes
`POST /v1/sessions`	Create a conversation scoped to a dataset and snapshot.	Returns session ID and capability card.
`POST /v1/sessions/{id}/messages`	Send a question; returns a plan, clarification, or unsupported response.	Streams `plan.*` events; never executes without confirmation.
`POST /v1/plans/{id}/confirm`	Confirm or edit parameters; starts a run.	Idempotency key required; returns run ID and event stream URL.
`POST /v1/plans/{id}/cancel`	Abandon a plan.	No run is created.
`GET /v1/runs/{id}`	Completed run payload: verdict, facts, tables, assumptions, warnings, exports.	Returns 409 while running if no partial payload exists.
`GET /v1/runs/{id}/events`	SSE stream of run stages.	Reconnect with `Last-Event-ID`.
`GET /v1/runs/{id}/trace`	Full audit trace and validation results.	Scope-checked; amounts included only for authorized roles.
`POST /v1/runs/{id}/reperform`	Re-execute the pinned tool and compare steps.	Queued; returns comparison result.
`GET /v1/runs/{id}/export?format=`	XLSX, PDF, or JSON export of the working paper.	Asynchronous for large files; signed URL.
`POST /v1/datasets`	Upload a template file or start a connector extract.	Returns load ID; quality report follows.
`GET /v1/datasets/{id}/quality`	Gate results, coverage, remediation links.	Same payload as the quality card.
`GET /v1/registry/accounts`	Search and list registry versions.	Filter by amount, status, behavior.
`PATCH /v1/registry/accounts/{id}`	Propose a new classification version.	Requires approval before use in a run.
`GET /v1/capabilities`	Supported analyses, limits, and dataset readiness for the tenant.	Used by UI and planner; no model inference.

8.2 Example: plan, confirm, run

```
POST /v1/sessions/ses_8a/messages
{ "text": "Custom motor order: 2,000 units at ₹380. We have spare capacity." }

200 OK
{
  "plan_id": "plan_5d",
  "status": "awaiting_confirmation",
  "plan": {
    "intent": "special_order",
    "tool": "run_differential_analysis",
    "arguments": { "decision_type": "special_order", "product_id": "PRD-B",
                   "order_units": "2000", "order_price": {"amount": "380.00", "currency": "INR"},
                   "capacity_state": "idle" },
    "confidence": 0.88
  },
  "parameter_sheet": [
    {"key": "product_id", "label": "Product", "value": "PRD-B Custom motor",
     "source": "lookup", "confirm": false},
    {"key": "capacity_state", "label": "Capacity", "value": "Idle",
     "source": "user_implied", "confirm": true,
     "warning": "September utilization was 92% for PLT-01. Confirm spare capacity."},
    {"key": "incremental_fixed_cost", "label": "Incremental fixed cost", "value": "Not provided",
     "source": "missing", "confirm": true}
  ],
  "missing": ["incremental_fixed_cost"]
}
```

```
POST /v1/plans/plan_5d/confirm
{ "plan_version": 1,
  "edits": [{"key": "capacity_state", "value": "idle", "reason": "spare shift documented"},
            {"key": "incremental_fixed_cost", "value": {"amount": "40000.00", "currency": "INR"}}] }

202 Accepted
{ "run_id": "run_01J9ZK4T7Q", "events": "/v1/runs/run_01J9ZK4T7Q/events" }
```

```
GET /v1/runs/run_01J9ZK4T7Q
{
  "status": "completed",
  "verdict": {"decision": "accept", "conditions": 4},
  "facts": [{"id": "f1", "label": "Incremental profit", "value": "120000.00", "unit": "INR"}],
  "tables": [{"id": "differential", "rows": 8}],
  "assumptions": [{"id": "a1", "text": "Idle capacity exists", "confirmed": true}],
  "audit": {"trace": "/v1/runs/run_01J9ZK4T7Q/trace",
            "reperform": "/v1/runs/run_01J9ZK4T7Q/reperform"},
  "exports": {"xlsx": "…", "pdf": "…", "json": "…"}
}
```

8.3 Streaming events

Stage events, not partial prose: a token stream of an unvalidated rationale invites the user to read numbers before they are checked. The UI shows progress against the state machine and renders the card once validation and rendering complete.

Event	Payload
`plan.created`	Plan ID, confidence, tool, missing fields.
`clarification.required`	One question, suggested answers, fields unblocked.
`confirmation.required`	Parameter sheet payload.
`run.started`	Run ID, pinned snapshot and versions.
`step.completed`	Step ID, label, duration. No values until validated.
`run.validated`	Validation summary; warnings count.
`run.completed`	Full card payload and export links.
`run.failed`	Error envelope with remediation actions.
`heartbeat`	Keep-alive every 15 seconds.

8.4 Error taxonomy

Every error has a code, a cause, what was checked, and executable remediation. The client renders remediation as buttons, not as a sentence the user must translate into an action.

Code	HTTP	Meaning	Remediation actions
`E_PLAN_AMBIGUOUS`	200	Plan confidence below threshold or dimensions ambiguous.	`choose_interpretation`, `select_dimension`
`E_PLAN_UNSUPPORTED`	200	Intent outside the capability card.	`show_supported`, `nearest_analysis`
`E_PLAN_SCHEMA`	422	Planner output failed schema or policy.	`retry_plan`, `open_form`
`E_ASSUMPTION_UNCONFIRMED`	409	Policy-bearing assumption lacks confirmation.	`open_parameter_sheet`
`E_DATA_MISSING`	409	Mandatory fact not loaded.	`upload_template`, `exclude_scope`

Code	HTTP	Meaning	Remediation actions
`E_DATA_QUALITY`	409	Blocking gate failed.	`fix_account_tag`, `view_gate`, `reload_file`
`E_ENGINE_DOMAIN`	422	Mathematical or policy precondition violated.	`review_inputs`, `run_alternative_view`
`E_VALIDATION_PROVENANCE`	500	Rendered narrative failed provenance after retry.	`view_trace`, `report_issue`; template fallback already used.
`E_LLM_UNAVAILABLE`	503	Planner or renderer unavailable.	`open_form`, `tables_only`, `retry`
`E_AUTH_SCOPE`	403	Requested scope outside the user's role.	`request_access`, `change_scope`
`E_EXPORT_FAILED`	500	Export generation failed.	`retry`, `alternate_format`
`E_RATE_LIMIT`	429	Tenant or user limit exceeded.	`retry_after`, `queue`

8.5 Exports

The export is a copy of a validated run, not a fresh calculation. It carries the run ID, snapshot ID, engine and rulebook versions, and the validation summary on every sheet or page.

- **XLSX:** tabs for Summary, Analysis, Trace, Assumptions, Sources, and Data Quality. Numbers are stored as numbers with explicit formats, not strings. Freeze panes and column widths are set. No macros.
- **Formula-injection protection:** any text cell beginning with `=`, `+`, `-`, `@`, tab, or carriage return is prefixed with an apostrophe. This applies to account names, product names, and every imported string.
- **PDF:** server-rendered from the same structured payload as the card, not a screenshot. Page footer includes run ID and "Draft" status until the run is sealed.
- **JSON:** the full trace schema for audit tooling and re-performance.

8.6 Frontend surface

Component	Responsibility
`ChatComposer`	Text input, dataset/snapshot context, suggested questions from capabilities.
`PlanCard`	What will run, which tool, which fields are missing; no verdict.
`ParameterSheet`	Argument values, source badges, edits, warnings, confirmation.
`AnalysisCard`	Verdict, KPI strip, tables, waterfall, assumptions, audit drawer, actions.
`FinanceTable`	Right-aligned tabular figures, F/A labels, row-level source links, no client arithmetic.
`VarianceWaterfall`	Renders engine-supplied bar values; no derived heights.
`AssumptionChips`	Source, confirmation, edit, and "what changes if…" link.
`AuditDrawer`	Steps, rules, formulas, sources, validation, re-perform.
`RemediationCard`	Error cause, checked items, action buttons.
`RegistryGrid`	Bulk classification sorted by amount, evidence panel, approval workflow.

Client state is server state. Financial values are fetched, never computed. Percentages, subtotals, and differences shown in the interface are fields in the payload. If a field is missing, the UI shows a dash and a link to the trace, not a client-side calculation.

9 Repository and engineering standards

9.1 Monorepo layout

```
costpilot/
  apps/web/                    Next.js application
  services/api/                FastAPI app, auth, sessions, runs, streaming
  services/worker/             ingestion, snapshots, exports, monitoring, re-performance
  packages/contracts/          Pydantic models → JSON Schema → TypeScript types
  packages/planner/            prompts, planner client, plan validation, policy
  packages/renderer/           verdict templates, formatting, slot substitution, exports
  packages/audit/              trace models, hash chain, sealing, verification
  packages/data/               schema DDL, ingestion, quality gates, query compiler
  packages/engine/             pure calculation library (Section 6.2)
  tests/golden/                YAML golden cases and expected traces
  tests/property/              Hypothesis identity and metamorphic tests
  tests/integration/           fixtures → snapshot → tool → trace → export
  evals/                       planner datasets, scoring, red-team suite
  deploy/                      Dockerfiles, compose, migrations, runbooks
  docs/                        ADRs, rulebook catalogue, this blueprint
```

The import graph is enforced: `engine` imports only standard library and its own modules; `renderer` imports contracts and formatting utilities, never the database; `planner` imports contracts, never the engine; `api` may orchestrate but may not contain formulas. An import-linter contract fails CI on violation.

9.2 Stack decisions

Layer	Choice	Rationale and rejected alternative
API	Python 3.12, FastAPI, Pydantic v2	Async, typed, OpenAPI. Rejected Django: heavier and ORM-centric for an analytics API.
Analytical store	DuckDB over immutable Parquet snapshots	Zero-ops columnar engine, fast scans, easy per-snapshot isolation. Postgres remains the metadata store and the migration path for very large deployments.
Metadata store	PostgreSQL 16, SQL-first with Alembic migrations	Audit integrity, transactions, row-level access. Rejected ORM-heavy access for analytics; metadata only.
Jobs	Redis 7 with RQ	Simple, observable, adequate for ingestion and exports. Rejected Celery for operational overhead at pilot scale.
LLM access	Provider SDK behind a gateway	No agent framework in the hot path. LangChain-style orchestration hides prompts, retries, and tool boundaries; the orchestrator is deterministic and must stay inspectable.
Frontend	Next.js 15, React 19, TypeScript, Tailwind	Server-rendered shell, typed client, fast iteration. Charts with visx so no library computes values.
Contracts	Pydantic → JSON Schema → generated TypeScript	One source of truth for tools, API, forms, and client types. Contract diffs fail CI.
Testing	pytest, Hypothesis, Playwright	Formula correctness, invariants, end-to-end flows. Golden cases are data, not code.
Packaging	uv, Docker, GitHub Actions	Reproducible builds and fast CI. Signed images and SBOM at release.
Observability	OpenTelemetry, structured logs, Prometheus metrics	Run-level tracing without logging amounts by default.
Exports	openpyxl, headless Chromium print	Native numeric cells and formula-injection protection; PDF from the same payload.

9.3 Coding standards

- **Engine purity.** No I/O, no database, no model, no clock, no random. Inputs and outputs are frozen dataclasses. Every rule has a docstring with its rule ID, source citation, and a worked example.
- **No floats.** A lint rule bans `float` in engine code. Decimal is constructed from strings. Division contexts are explicit.
- **Typed boundaries.** `mypy --strict` on contracts, engine, planner, and renderer. The API may use narrower typing at the framework edge only.
- **SQL discipline.** Parameterized queries only; no string interpolation of user values. Query compiler owns scope predicates. Every analytical query is reviewed and has a test fixture.
- **Errors are typed.** Domain exceptions carry a code and remediation. No bare `Exception` reaches the API boundary.
- **Logs exclude amounts by default.** Log IDs, hashes, stages, durations, and gate results. Amounts are available in the trace to authorized users, not in application logs.
- **Version everything that changes an answer:** engine, rulebook, registry, tool, prompt, contract schema, snapshot transform. Versions are semantic and stored in the run.
- **ADRs for structural changes.** Data model, trust boundary, tool contract, and trace schema changes require a short architecture decision record.

9.4 CI gates

Stage	Gate
Pull request	Lint, strict types, unit tests, fast golden subset, contract compatibility diff, import-boundary check, dependency and secret scan, frontend build and component tests.
Nightly	Full golden corpus, property and metamorphic suite, live planner eval, performance smoke, re-performance sample, accessibility scan.
Release	Migration up/down test, full E2E, load test at pilot concurrency, backup and restore drill, SBOM and signed image, runbook review.

No merge with a failing golden case. No release with a provenance or verdict-fidelity failure. A performance regression above 20% on the decision path requires an explicit waiver with an owner and a date.

9.5 Observability and SLOs

Signal	Target or use
Decision answer latency	p95 under 10 seconds end to end; planner under 3 seconds; engine under 300 ms; render under 3 seconds.
Variance pack latency	p95 under 60 seconds for one million GL rows in scope.
Run success rate	Above 99% excluding user-correctable data errors; every failure carries a remediation.
Provenance failure rate	Below 0.5%; any sustained breach is a model-quality incident.
Re-performance mismatch	Zero. Any mismatch blocks release and triggers an audit.
Ingestion success	Above 99% of valid files; quality-gate pass rate tracked by customer.
LLM cost and tokens	Tracked per run and per tenant; pilot target under $0.10 per completed analysis.
Availability	99.5% pilot business hours; planned maintenance excluded.

Spans cover each state-machine stage. Logs carry `run_id`, `tenant_id`, `stage`, `tool`, `rule_version`, and `latency_ms`. Alerts fire on validator failure rate, queue backlog, snapshot build failure, audit write failure, and p95 latency breach. Audit write failure fails the run closed: no answer without a sealed record.

9.6 Security and privacy

Threat	Control
Cross-tenant or cross-plant access	OIDC authentication; roles viewer, analyst, controller, admin; row scope injected into SQL; per-tenant snapshots; authorization matrix tests.
Prompt injection through imported text	Account, product, and vendor strings are delimited as data; the planner receives no raw rows; data text can never change policy or trigger a tool; suspicious strings flagged.
Data exfiltration through the model	Metadata-only planner context; facts-only narration; zero-retention, no-training provider terms; region pinning; redacted request logging.
Malicious uploads	Type allowlist, size limit, macro rejection, zip-bomb protection, virus scan, staging isolation.
Spreadsheet formula injection on export	Prefix dangerous cell starts; escape strings; no macros; numeric cells typed.
Connector overreach	Read-only least-privilege credentials; no write scopes; egress allowlist; control-total reconciliation.
Audit tampering	Append-only tables, hash chain, object lock on sealed exports, verification endpoint.
Secret leakage	Vault or managed secrets; no secrets in env files committed; dependency and secret scanning.
Model error presented as fact	Slot protocol, verdict templates, validator, template fallback, re-performance.

10 Test and evaluation strategy

Correctness is demonstrated, not asserted. The test strategy has two independent axes: deterministic tests that prove the engine and data layer, and model evaluations that prove the planner and narration contract. A passing engine suite with a failing model eval is not a shippable product; neither is the reverse.

10.1 Test layers

Layer	Scope	Gate
Rule units	Every formula, sign, zero, negative, rounding, and missing-input case.	100% golden match; 95% branch coverage on engine.
Property tests	Identities such as MCV = MPV + MUV, bridge ties, ABC sums, flexible-budget ties, break-even yields zero income.	No counterexample in 1,000 generated cases per property.
Metamorphic tests	Scaling, price-only changes, quantity-only changes, reallocation, actual equals budget.	Expected monotonic or exact response.
Contract tests	Tool schemas, API payloads, TypeScript types, trace schema, backward compatibility.	No breaking diff without a version bump.
Integration	Fixture → ingestion → gates → snapshot → tool → trace → export.	End-to-end fixture passes for every workflow.
Planner evals	Intent, parameters, clarification, missing data, unsupported intent, injection.	Thresholds below.
E2E	Browser flows: ask, confirm, view, audit, re-perform, export, fix a gate.	Critical flows green on every release.
Performance	Decision latency, variance pack, ingestion, concurrency, memory.	Budgets in Section 9.5.

10.2 Golden corpus

Golden cases are the product's specification in executable form. Every worked example in the reference curriculum becomes a case, plus variants for edge conditions. Cases are YAML, versioned with the rulebook, and reviewed by the cost-accounting SME.

```yaml
id: VAR-MAT-001
module: variance.material
rule: MA-VAR-001
source: "Curriculum, Unit 7, worked example 2"
inputs:
  standard_price: "50.00"
  actual_price: "52.00"
  actual_quantity_used: "20500"
  standard_quantity_allowed: "20000"
expected:
  mpv: "-41000.00"
  muv: "-25000.00"
  mcv: "-66000.00"
  favorable: false
assertions:
  identities: ["mcv == mpv + muv"]
  trace_steps: 3
  units: ["INR", "INR", "INR"]
```

Coverage of the corpus: material, labour, variable and fixed overhead, flexible budget, sales price and volume, all seven decision workflows with at least three variants each (idle/constrained, profitable/unprofitable, missing/complete data), the costing bridge in both inventory directions, CVP at zero and negative contribution, and Phase 2 ABC, budget chain, ROI, and residual income. The corpus grows with every defect: a bug fix without a regression case is incomplete.

10.3 Property and metamorphic tests

- Scaling all volumes and variable costs by k scales contribution by k and leaves fixed cost unchanged.
- A change in actual price moves MPV and leaves MUV unchanged when quantity is held constant.
- A change in actual quantity moves MUV and leaves MPV unchanged when price is held constant.
- At the computed floor price, incremental profit is exactly zero.
- At the shutdown indifference point, operate and shut down profit are equal.
- When actual volume equals budget volume, the flexible budget equals the static budget and the volume variance is zero.
- Reallocating an ABC pool across drivers preserves the total allocated plus unused capacity.
- For any inventory increase, absorption income minus variable income equals change in units × fixed overhead rate.
- Rounding the display never changes the stored step value or the re-performance result.

10.4 Planner and narration evaluations

Metric	Threshold	Dataset
Top-1 intent accuracy	≥ 97%	250 questions across seven workflows, variance, bridge, and unsupported intents.
Required-argument exactness	≥ 98%	Same set, scored field by field, including units and currency.
Appropriate clarification	≥ 95%	80 ambiguous or incomplete questions.
Missing-data detection	≥ 98%	60 questions with a deliberately absent driver or standard.
Unsupported-intent refusal	≥ 95%	40 out-of-scope requests, including requests to estimate.
Uncited numeric claims	0	Every generated rationale in the eval set.
Verdict fidelity	100%	Every decision case; the rendered verdict must equal the engine verdict.
Injection resistance	100%	30 adversarial rows, questions, and account names; no policy change or tool escalation.

Metric	Threshold	Dataset
Planner p95 latency	≤ 3 s	Production-like model endpoint.

Evals run against recorded fixtures on every pull request and against the live model nightly. A model or prompt change that fails a threshold cannot ship. Scores are stored with the prompt hash so regressions are attributable.

10.5 Data-quality and ingestion tests

- Fixture per gate: schema mismatch, orphan account, missing period, duplicate hash, control-total break, low coverage, missing UOM, mixed currency, sign anomaly, outlier, missing standard card, missing usage.
- Blocking gates must prevent snapshot creation and produce a remediation card. Warnings must not block.
- Unknown dimension members must land in the mapping queue, never in canonical facts.
- Re-loading a corrected file creates a new snapshot with lineage; the prior snapshot remains byte-identical.
- Snapshot manifest re-hashing reproduces the snapshot ID.

10.6 Performance and reliability tests

The performance fixture is one million GL rows across two plants and three products, plus standards, budgets, and volumes. Tests measure cold and warm snapshot queries, decision latency, variance-pack generation, concurrent analyst sessions, memory per worker, and export time. Reliability tests cover worker crash during ingestion (no partial canonical load), snapshot corruption (rebuild from Parquet), audit write failure (run fails closed), and model outage (workflow lane and tables-only mode).

10.7 Pilot acceptance

- ☐ A controller re-performs five answers, one from each workflow family, without engineering help.
- ☐ The pilot team reproduces one real close-cycle variance pack end to end and uses it as the working paper.
- ☐ No answer in the pilot contains an uncited number or an unconfirmed policy assumption.
- ☐ The special-order analysis returns in under ten seconds and matches a hand-built working paper.
- ☐ Every blocked analysis produces a remediation that the pilot team can complete without a support call.
- ☐ Export opens in Excel with live numbers, correct formats, and no formula execution from imported strings.

11 Delivery roadmap

The milestone sequence follows the dependency chain, not the sales narrative. Variance and audit come before chat because the chat shell is only useful once the tools are trustworthy. Decision analysis comes before the planner for the same reason, and because it is the strongest end-to-end demonstration. Durations below are indicative for a team of three engineers, one part-time frontend engineer, and a cost-accounting advisor; they assume the synthetic dataset is ready at M0.

11.1 Phase 1 sequence

Milestone	Time	Contents	Exit gate
M0 Foundations	1 wk	Repository, CI, contracts, trace models, Decimal utilities, rule registry skeleton, golden harness, three-product/two-plant fixture, ADRs for trust boundary and data model.	CI green; one dummy rule executes with a complete trace; contracts and schema v0 reviewed.
M1 Data + formulas	3 wks	Canonical schema, ingestion, quality gates, account registry, snapshot builder, material/labour/overhead formulas, flexible budget.	All Unit 7/8 golden cases pass; fixture reconciles to control totals; weighted coverage ≥ 99%; manifest re-hash reproduces the snapshot ID.

Milestone	Time	Contents	Exit gate
M2 Variance + audit	3 wks	Account-level flux, standard-cost explainer, drill-down, cause catalogue, materiality thresholds, costing bridge, slot narration, audit drawer, XLSX export.	One month of actuals produces a close-ready pack; every fact cited; three controller re-performances pass; export opens with live numbers.
M3 Decision engine	3 wks	Seven workflows, parameter confirmation, qualitative checklists, sensitivity, capacity conflict checks, verdict templates.	Special order returns in under 10 s; 21 decision golden cases pass; floor price and sensitivity exact; zero uncited numbers.
M4 Planner + shell	3 wks	Planner prompt and client, metadata tools, sessions, SSE, parameter sheet, generated workflow forms, RBAC, remediation cards, PDF/JSON export.	A non-accountant completes ten tasks unassisted; intent ≥ 97%; injection suite 100%; cross-scope access denied; every error has an action.
M5 Pilot	2–3 wks	Hardening, load test, runbooks, documentation, one live close-cycle trial, defect burn-down.	Pilot acceptance checklist complete; no open P1; SLOs met for two consecutive weeks; controller sign-off on audit.

11.2 Critical path and demonstration

The critical path is **M0 → M1 → M2 → M4 → M5**. M3 runs in parallel with M2 once M1 lands, using the same engine contracts and a different fixture. The first sellable demonstration is M3's special-order answer running end to end with confirmation, audit, and export. The first operational proof is M2's close-ready variance pack, because it exercises ingestion, gates, flexible budget, narration, and export on messy data.

Two dates matter more than the rest. The **M1 gate** decides whether the data layer is real; if coverage or reconciliation is weak, stop and fix it rather than building UI on sand. The **M2 gate** decides whether the narration protocol works under production variance data; if the slot validator is noisy, fix it before adding seven more workflows.

11.3 Team shape

Role	Allocation	Responsibility
Technical lead / backend	1.0	Orchestrator, contracts, security, data layer, integration, release.
Engine / data engineer	1.0	Rulebook, engine modules, golden corpus, quality gates, snapshots.
Frontend engineer	1.0	Chat, parameter sheet, analysis cards, audit drawer, registry grid, exports UI.
Cost-accounting advisor	0.4–0.5	Rulebook review, golden cases, cause catalogue, pilot data readiness, controller acceptance.
Product / design	0.25	Scope control, card information hierarchy, pilot coordination.
DevOps / security	0.25	Deployment, secrets, backups, observability, security review.

The advisor is not optional. Without a cost accountant reviewing the rulebook and golden cases weekly, the team will encode formulas that look plausible and fail a controller's first test — particularly in overhead sign conventions, relevant-cost treatment, and semi-variable splits.

11.4 Phase 2 sequence and extension points

Milestone	Time	Contents	Exit gate
M6 Connectors	4 wks	SAP or NetSuite read connector, account-mapping workflow, ingestion quality reports, control totals, incremental extraction.	A live extract loads into the registry and reconciles to ERP control totals; mappings approved by a human.
M7 ABC / TDABC	4 wks	Cost pools, drivers, driver events, activity rates, allocation, traditional-versus-ABC profitability, TDABC capacity rates, customer profitability.	Reproduces hidden undercosting of low-volume products on pilot data; allocations tie to pools; unused capacity reported separately.

Milestone	Time	Contents	Exit gate
M8 Monitoring	2 wks	Rolling variance and anomaly detection, materiality-ranked digest, natural-language alert rules compiled to engine queries.	Pilot team receives a weekly exception digest with no manual report run; alerts link to full traces.
M9 Budgeting	4 wks	Sales-to-cash budget chain, versioned consolidation, scenario overlays, rolling reforecast.	One full reforecast in days rather than weeks; version differences explainable by the engine.
M10 Performance	2–3 wks	ROI and residual income comparison, controllability tagging, lightweight auto-populated scorecard.	Divisional review runs on copilot output; no parallel spreadsheet.
M11 Group + hardening	4 wks	Multi-entity bridges, report-pack export, RBAC expansion, governance polish, intercompany flags.	Phase 1 and Phase 2 pilot team operates weekly without CSV fallback.

Dependencies. M6 precedes M7, M8, and M10 because they need richer ingestion. M9 can run in parallel with M7 once M6 is stable. M11 follows M7–M10. The Phase 1 trace, tool envelope, and snapshot manifest are unchanged in Phase 2; new tools add new result types, not a second audit mechanism.

Extension points designed now, not later. The tool registry is versioned so Phase 2 tools can be added without breaking the planner. The snapshot schema is additive, with driver events as new facts rather than a parallel store. The executor treats a monitor run exactly like a user run, with `origin=monitor`, so alerts inherit the same trace and validation. The connector interface is defined as discover, extract, control-totals, and map, with human approval on mappings. Budget versions and scenario overlays are new tables keyed by the same period and account dimensions. No Phase 1 module may assume a single product, plant, or period in a way that blocks multi-entity reporting.

11.5 Change control

Four artifacts are under change control: the tool contract, the trace schema, the data schema, and the rulebook. A change to any of them requires an ADR, a version bump, a migration plan, and the relevant eval or golden suite. Prompt changes require the planner eval subset. A change to the scope contract in Section 12.3 requires explicit product-owner and engineering-lead approval; it is not a backlog decision.

12 Risks and open decisions

12.1 Build risks

Risk	Severity	Mitigation
Pilot cost data is untagged or unreconciled	HIGH	Treat the registry as M1 scope; run a data-readiness assessment before signing a pilot; make remediation a product surface; choose a pilot with a named data owner and 12 months of history.
The planner chooses the wrong workflow or misses a parameter	HIGH	Confirmation sheet, confidence threshold, metadata-only tools, 250-question eval, workflow form fallback, and a planner that can say “unsupported”.
Narration validator becomes brittle and blocks answers	MEDIUM	Build the slot protocol at M2, not M4; keep the numeric rule simple; use template fallback rather than repeated regeneration; track failure rate as a model-quality metric.
Rule drift between engine, documentation, and UI	MEDIUM	Single rule registry, generated rulebook catalogue, rule IDs in every trace, golden cases reviewed by the SME, no formulas outside the engine.
DuckDB or snapshot performance degrades at scale	MEDIUM	Benchmark at M1 with one million rows; pre-aggregate marts; memory caps per worker; keep a Postgres/columnar migration path; avoid per-row Python loops.
Scope creeps toward ERP, consolidation, or ABC in Phase 1	MEDIUM	Out-of-scope contract; milestone gates; change control; defer connector and ABC requests to M6/M7 even when a pilot asks.
Users trust a conditional verdict as unconditional	HIGH	Verdict templates encode conditions; assumptions are visible chips; unresolved qualitative checks make the verdict conditional; export repeats the conditions.

Risk	Severity	Mitigation
Model provider changes, deprecates, or fails residency review	MEDIUM	Provider abstraction, pinned model versions, eval suite portable across providers, region-pinned zero-retention terms, workflow lane works without the model.
Audit storage and trace size grow faster than expected	MEDIUM	Compress traces; store facts and steps, not raw query results; separate hot metadata from cold exports; seven-year retention with tiering.
Key-person dependence on the cost-accounting advisor	MEDIUM	Rulebook as written specification; golden cases encode the advisor's judgement; two reviewers on rule changes; record policy decisions as ADRs.

12.2 Open decisions and defaults

These are the choices this blueprint makes provisionally. Each needs an owner and a decision date; until then, the default applies and is recorded in the run.

Decision	Default	Owner	By
LLM provider and residency	Frontier model via a zero-retention, region-pinned gateway; provider abstraction from day one.	Eng lead + security	M0
Hosting model	Single-tenant managed private cloud; customer VPC as an option for the pilot.	Eng lead	M2
Authentication	OIDC with the customer's identity provider; local accounts only for development.	Eng lead	M4
Queue and job runner	Redis with RQ; no Celery at pilot scale.	Eng lead	M4
Inventory costing method	Standard costing only in Phase 1; FIFO and weighted average flagged as unsupported.	Cost advisor	M1
Material price variance basis	Usage basis for tie-out; purchase-basis variance shown as a supplement when quantities differ.	Cost advisor	M1
Semi-variable split	User-specified split or high-low over at least six periods; regression in Phase 2.	Cost advisor	M1
Rounding and display	Full precision in trace; half-up display; currency 2, rates and quantities 4, percentages 2; visible rounding adjustment.	Eng lead	M1
Currency scope	One functional currency per entity; no FX translation in Phase 1.	Product	M1
Cause catalogue ownership	SME-curated catalogue keyed to variance patterns; the model phrases causes, never invents them.	Cost advisor	M2
Data retention	Audit metadata seven years; raw snapshots thirteen months in the pilot; contractual override allowed.	Product + security	M5
ABC allocation method	Direct and step-down in Phase 2; reciprocal allocation deferred to Phase 3.	Cost advisor	M7
Budget scope	Template chain, versioned consolidation, and scenarios only; no form-based budget capture UI.	Product	M9
Scorecard weights	Client-owned, no defaults; weights are versioned and visible per result.	Product	M10

12.3 Phase 1 scope contract

The following are out of scope. They are not "later if there is time"; they are deferred with a destination, and reopening any of them requires explicit approval.

Excluded	Reason	Destination
ABC / TDABC product and customer profitability	Needs driver and activity data most pilot companies cannot supply quickly.	Phase 2, M7
Budgeting workflows and rolling forecasts	Organizational process change more than software; needs versioned chain.	Phase 2, M9
Performance measurement and scorecards	Secondary to the close-pain wedge; politically sensitive.	Phase 2, M10
ERP connectors	Integration effort is large; template ingress proves the concept.	Phase 2, M6
Multi-currency, translation, intercompany, consolidation	Enterprise rollout concerns, not concept proof.	Phase 2 M11 / Phase 3
Write-back to ledgers or ERP	Trust and security; the product is read-only by design.	Not planned
Statutory reporting, tax, audit opinion	Different discipline and liability profile.	Not planned
Marketing decision analytics	No domain knowledge layer in the curriculum yet.	Phase 3
ML demand forecasting	Forecasting is not the wedge; scenarios are deterministic.	Phase 3 or partner
Native mobile app	Responsive web is sufficient for analysts.	Not planned

12.4 Release definition of done

- ☐ All golden cases pass; no skipped or quarantined rule.
- ☐ Zero provenance failures and zero verdict-fidelity failures in the release eval.
- ☐ Re-performance sample is 100% exact; any display-rounding differences are documented.
- ☐ Planner eval thresholds met on the release model and prompt hash.
- ☐ Performance budgets met on the one-million-row fixture.
- ☐ Security scan clean of high and critical findings; export injection test passes.
- ☐ Audit hash chain verifies for the release fixture runs.
- ☐ Every error code has an actionable remediation in the UI.
- ☐ Runbooks updated for model outage, queue backlog, snapshot rebuild, and audit failure.
- ☐ Cost-accounting advisor signs the rulebook diff and the pilot acceptance checklist.

13 Appendices

A. Glossary

Term	Definition
ABC	Activity-based costing: allocating indirect costs through activity pools and drivers rather than volume bases.
TDABC	Time-driven ABC: capacity cost rate multiplied by time per transaction; unused capacity is measured explicitly.
MPV / MUV / MCV	Material price, usage, and total cost variances. MCV = MPV + MUV under the usage basis.
LRV / LEV / LCV	Labour rate, efficiency, and total cost variances.
VOH / FOH	Variable and fixed manufacturing overhead.
Flexible budget	A budget restated at actual volume, separating volume effects from spending, price, and efficiency effects.

Term	Definition
Contribution margin	Revenue minus variable costs; the amount available to cover fixed costs and profit.
CVP	Cost-volume-profit analysis: break-even, target profit, margin of safety, operating leverage.
Relevant cost	A future cost that differs between alternatives. Sunk costs and unavoidable allocated costs are excluded.
Opportunity cost	Contribution forgone from the best alternative use of a scarce resource.
Avoidable cost	A fixed cost that disappears if an activity, product, or facility is removed.
Standard cost card	Per-unit standards for material, labour, and overhead used in product costing and variance analysis.
Snapshot	An immutable, hashed set of canonical facts plus a registry and schema version, pinned by a run.
Step / fact	A calculation step is a rule application with inputs and a value; a fact is the render-facing projection of a step.
Rule ID	A stable identifier for a formula or decision policy, versioned and cited in the trace.
Provenance	The chain from a displayed number to a fact, a step, a rule, and a source record.
Re-performance	Re-executing a pinned run against its snapshot and code version and comparing every step value.

B. Contract snippets

Tool result envelope

```
interface ToolEnvelope {
  run_id: string;
  status: "ok" | "blocked" | "error";
  snapshot_id: string;
  result: Record<string, unknown>;
  facts: Fact[];
  assumptions: Assumption[];
  warnings: Warning[];
  remediation: RemediationAction[];
  trace_ref: string;
}

interface Fact {
  fact_id: string;          // "f1"
  step_id: string;          // "s6"
  label: string;
  value: string;            // exact decimal string
  unit: string;             // "INR", "INR/unit", "units"
  format: "INR" | "INR0" | "PCT1" | "PCT2" | "QTY0" | "QTY2" | "RATIO2" | "HOURS1";
}
```

Plan and error types

```
interface Plan {
  intent: Intent;
  tool: string;
  tool_version: string;
  arguments: Record<string, unknown>;
  assumptions: Assumption[];
  missing: string[];
  clarification: Clarification | null;
  confidence: number;
}

interface ErrorEnvelope {
  code: ErrorCode;
  message: string;
  cause: string;
  checked: string[];
  remediation: RemediationAction[];
}

interface RemediationAction {
  action: string;           // "fix_account_tag", "upload_template", "open_parameter_sheet"
  label: string;
  deep_link?: string;
  payload?: Record<string, unknown>;
}
```

C. Audit trace schema

Field	Type	Notes
`run_id`	string	ULID; stable across retries of the same confirmed plan.
`origin`	enum	`user`, `monitor`, `reperformance`.
`question`	string	Original user text; redacted only if it contains secrets.
`plan`	object	Raw and normalized plan, version, confidence, clarification.
`confirmation`	object	Required flag, user, timestamp, proposed versus confirmed edits.
`pins`	object	Snapshot, manifest, registry, engine, rulebook, prompt, model, tool versions.
`arguments`	object	Resolved arguments with dimension labels; money as amount and currency.
`assumptions`	array	ID, text, source, confidence, confirmation, effect.
`steps`	array	Step ID, rule ID and version, formula, inputs, value, unit, source refs.
`facts`	array	Fact ID, step ID, label, exact value, unit, format.
`verdict`	object	Decision enum, template ID and version, conditions.
`rationale`	object	Raw placeholder text, rendered text, validation result, fallback flag.
`validation`	map	Schema, identities, units, basis, coverage, assumptions, provenance.
`hashes`	object	Input, output, previous event, event hash.
`latency_ms`	object	Plan, execute, validate, render, total.
`error`	object | null	Error envelope when the run failed.

D. Rulebook template and ID convention

Rule IDs use a module prefix and a three-digit sequence. The version is the rule's own semantic version, independent of the engine version. A rule change that can alter a number requires a major or minor bump and a new golden case.

Prefix	Scope	Example
`MA-VAR`	Material variances	`MA-VAR-001` material price, usage basis
`LA-VAR`	Labour variances	`LA-VAR-002` labour efficiency
`OH-VAR`	Overhead variances	`OH-VAR-004` fixed overhead volume
`FB-VAR`	Flexible budget	`FB-VAR-001` flexible-budget variance
`SAL-VAR`	Sales variances	`SAL-VAR-002` sales volume
`DA-*`	Differential analysis	`DA-SO-006` special-order incremental profit
`CVP`	Cost-volume-profit	`CVP-001` break-even units
`CB`	Costing bridge	`CB-001` absorption-versus-variable profit
`ABC`, `TDABC`	Activity costing	`ABC-003` pool rate; `TDABC-002` unused capacity
`BUD`	Budget chain	`BUD-002` material purchases
`PERF`	Performance	`PERF-002` residual income

```
id: DA-SO-006
version: 1.0.0
title: Special-order incremental profit
module: decisions.special_order
source: "Curriculum, relevant-cost chapter; policy decision ADR-014"
formula: revenue - variable_cost - incremental_fixed - opportunity_cost
inputs:
  order_units: quantity
  order_price: money_per_unit
  variable_cost_per_unit: money_per_unit
  incremental_fixed: money
  opportunity_cost: money
outputs:
  incremental_profit: money
  verdict: enum[accept, reject, investigate]
policy:
  exclude_allocated_fixed: true
  include_opportunity_cost: true
  conditional_if_unresolved_checklist: true
tests:
  - GOLDEN-DA-SO-001
  - GOLDEN-DA-SO-002
  - PROP-DA-SO-FLOOR
```

E. Source-to-blueprint traceability

Source requirement	Blueprint location	Milestone
Language model never performs arithmetic	§2 I1; §4.1, §4.6; §6.1	M0, M2, M4
Every answer carries an audit trail	§7; §8.2, §8.3	M2, M4
Account registry with fixed/variable and direct/indirect tags	§5.4	M1
Standard-cost variance decomposition	§6.4	M1, M2
Flexible-budget restatement	§6.4	M2

Source requirement	Blueprint location	Milestone
Seven short-run decision workflows	§6.5, §6.7	M3
Variable versus absorption costing bridge	§6.6	M2
ABC / TDABC and product profitability	§5.9, §6.8	M7
Budgeting, consolidation, rolling forecast	§5.9, §6.8	M9
ROI, residual income, lightweight scorecard	§6.8	M10
Read-only ERP connectors with governed mappings	§3.5, §5.9, §9.6	M6
Continuous monitoring and materiality-ranked alerts	§5.9, §6.8, §11.4	M8
Group-level bridges and intercompany flags	§6.8, §11.4	M11
Same question, same number	§2 I3; §7.3, §7.4	M1, M5
No silent assumptions	§2 I5; §4.5	M3, M4
No dead-end errors	§2 I9; §8.4	M4

F. Pilot data-readiness checklist

- ☐ Twelve months of GL actuals at account × plant × cost-center grain, with a control-total reconciliation per period.
- ☐ Chart of accounts with statement, type, and a named owner for classification decisions.
- ☐ Standard cost cards covering at least 90% of product cost, with material, labour, variable overhead, and fixed overhead components.
- ☐ Standard usage per product: material quantity, labour hours, machine hours, and UOM.
- ☐ Actual quantities, prices, hours, and rates, or a subledger extract from which they can be produced.
- ☐ Production, sales, and inventory movement by product and plant.
- ☐ Budget by account and period, with the version that was in force during the close.
- ☐ Capacity and utilization evidence for the plants in scope, sufficient to test an idle-capacity claim.
- ☐ For Phase 2: setup, inspection, purchase-order, and order-line event logs for at least one plant.
- ☐ A named data owner, a cost-accounting reviewer, a security review contact, and a pilot close date.

End of blueprint · Version 1.0 · November 2026 · Companion to the CostPilot White Paper, Phase 1 Plan, and Phase 2 Plan.